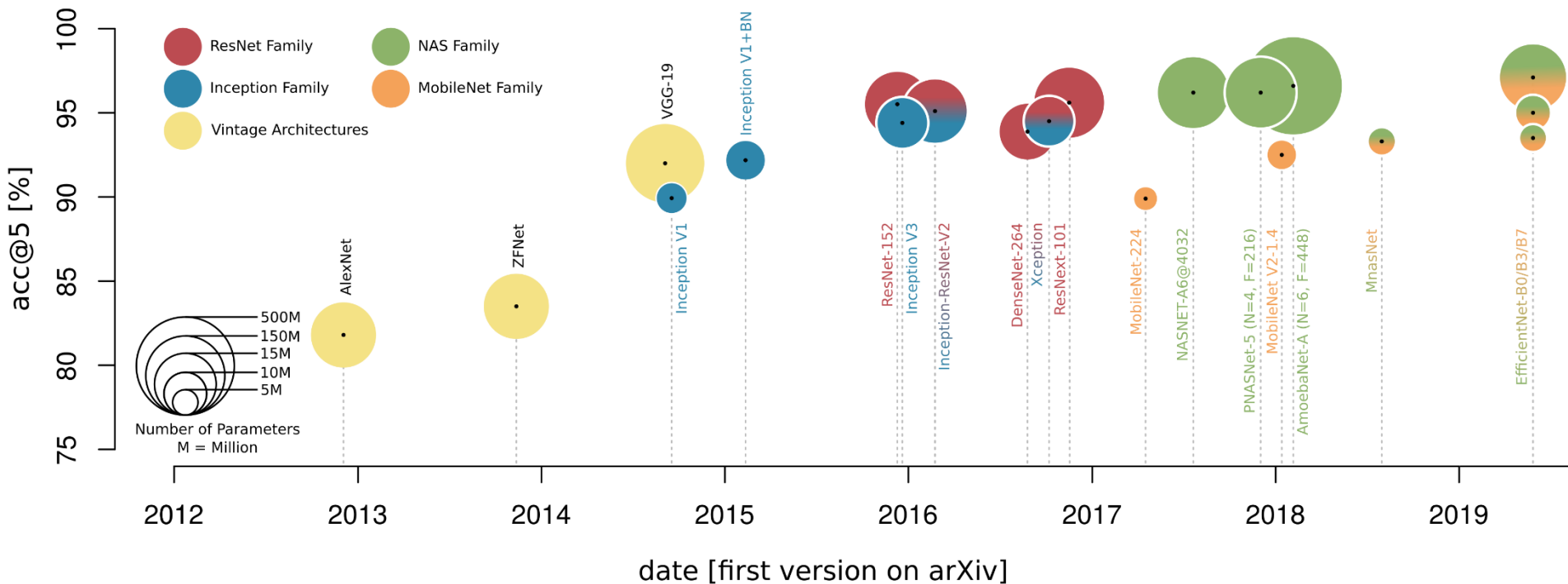


Deep Neural Networks and Convolution Neural Networks

Convolution Neural Networks (CNN)

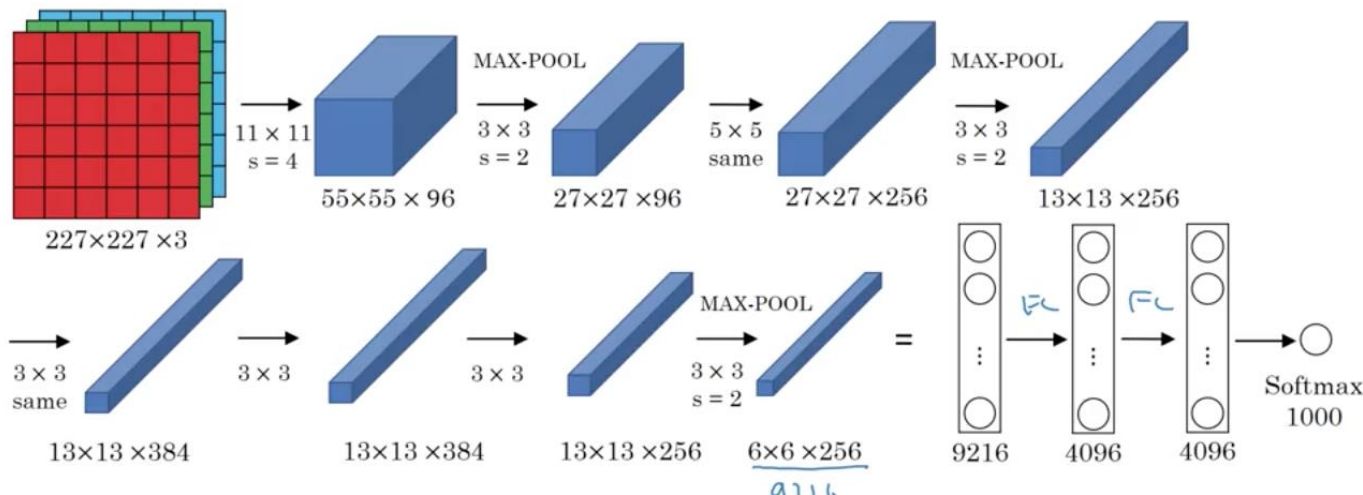
CNN 구조 (Architecture of CNN)



[Source] Hoer, T; Kuenzer, C. Object Detection and Image Segmentation with Deep Learning on Earth Observation Data: A Review-Part I: Evolution and Recent Trends. Remote Sensing 2020, 12(10), 1667.

Convolution Neural Networks (CNN)

AlexNet



[227x227x3] Input

[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

[27x27x256] **Conv2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization Layer

[13x13x384] **Conv3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **Conv4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **Conv5**: 256 3x3 filters at stride 1, pad 1

[6 x 6 x 256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

AlexNet (Hinton)은 2012년에 개최된

ILSVRC(**ImageNet** Large Scale Visual Recognition Challenge) 대회의 우승을 차지한 컨볼루션 신경망 (CNN) 구조이다. CNN의 부흥에 아주 큰 역할을 한 구조라고 말할 수 있다.

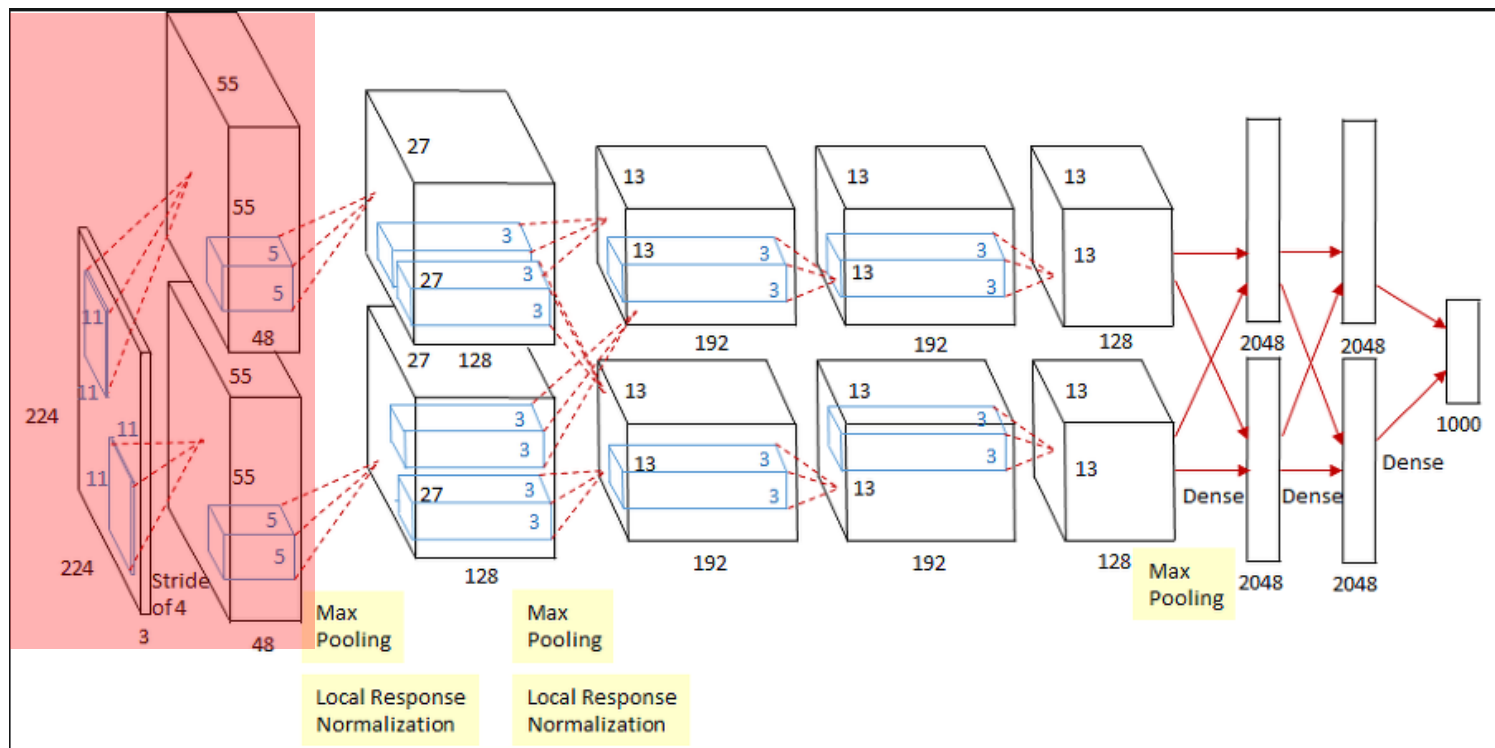
❖ 입력, 출력 224x224x3, 1000 class

❖ 파라미터 수 (60 million, 5 convolutional layers, 3 fully connected layers)

Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey. ImageNet classification with deep convolutional neural networks. In NIPS. 2012.

Convolution Neural Networks (CNN)

Alexnet 구조 (Architecture of CNN)



❖ 두 갈래의 길 – GPU 연산이 한번에 불가

- 첫 번째 이유는 GPU 사용 시 학습 속도 향상
- GTX580, 메모리 3GB → 메모리제한
- 각 48개 필터
- 현재는 이렇게 사용 안함

[227x227x3] Input

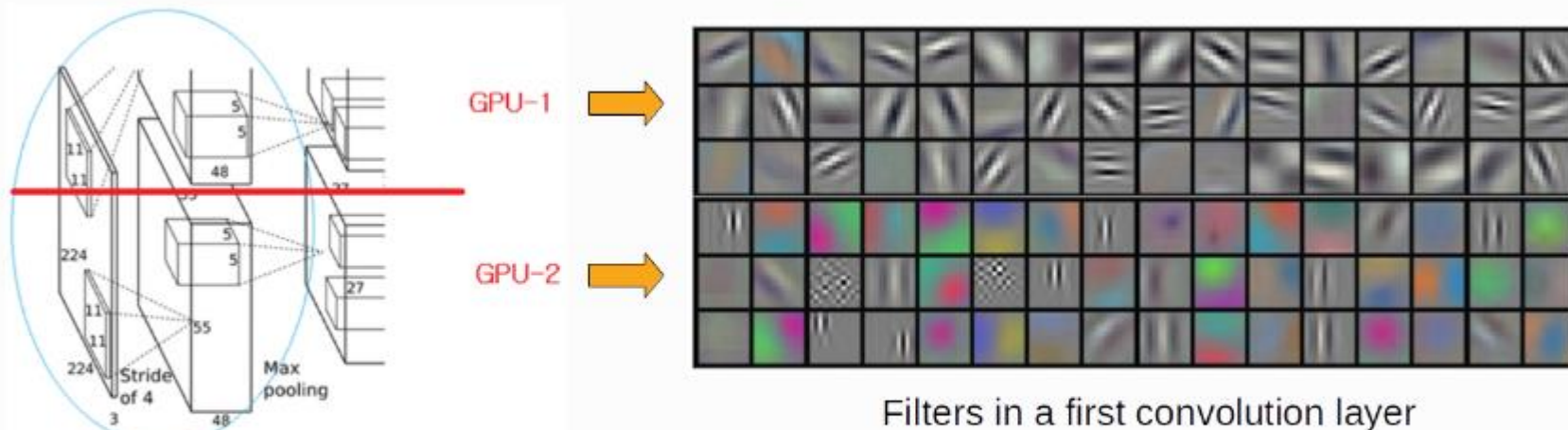
[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

Convolution Neural Networks (CNN)

Alexnet 구조 (Architecture of CNN)



출처: 라온피플 (Laon People)

Convolution layer에서 GPU-1은 주로 컬러와 상관없는 48개의 Filter로 학습됐고, GPU-2에서는 주로 Color와 관련된 48개의 Filter를 학습되었다고 한다.

❖ 두 갈래의 길 – GPU 연산이 한번에 불가

- 첫 번째 이유는 GPU 사용 시 학습 속도 향상
- GTX580, 메모리 3GB → 메모리제한
- 각 48개 필터
- 현재는 이렇게 사용 안함

[227x227x3] Input

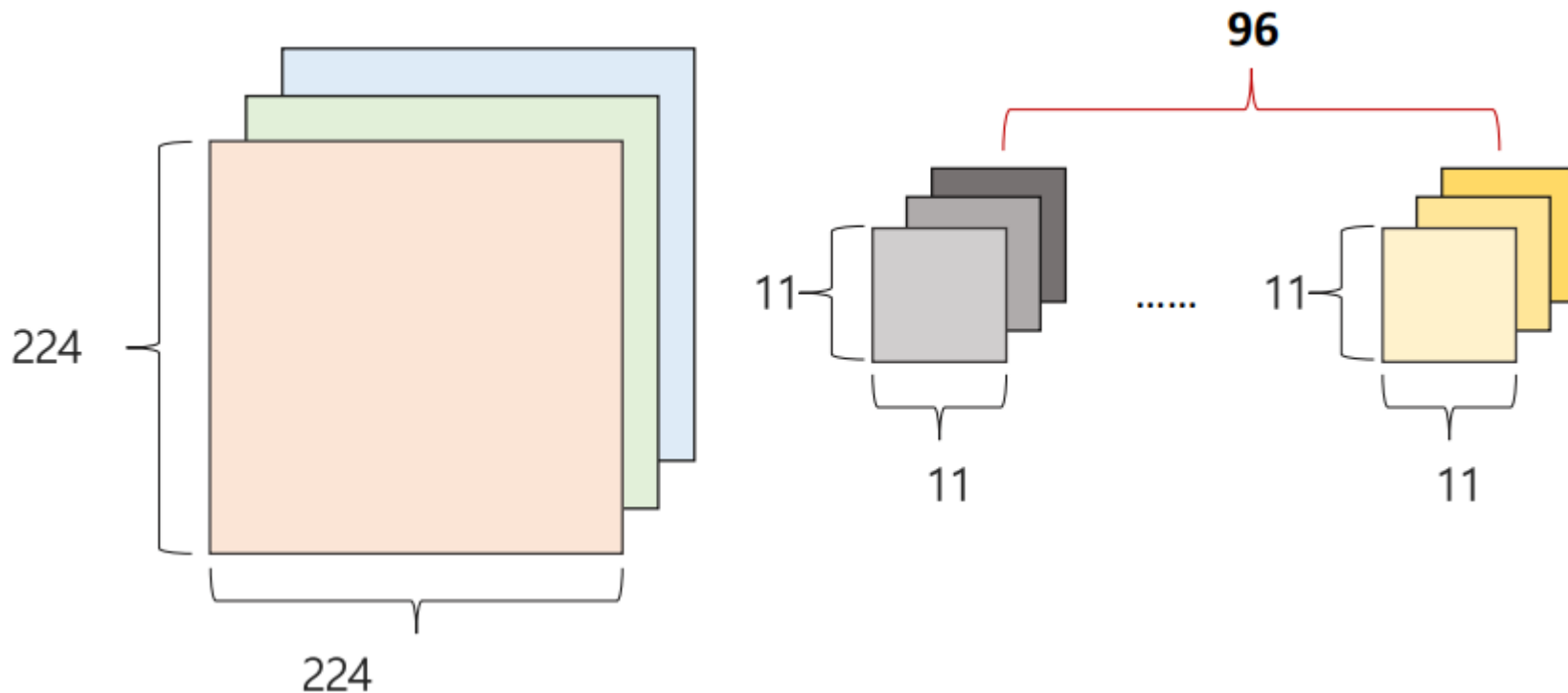
[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

Convolution Neural Networks (CNN)

Alexnet 구조 (Architecture of CNN)



[227x227x3] Input

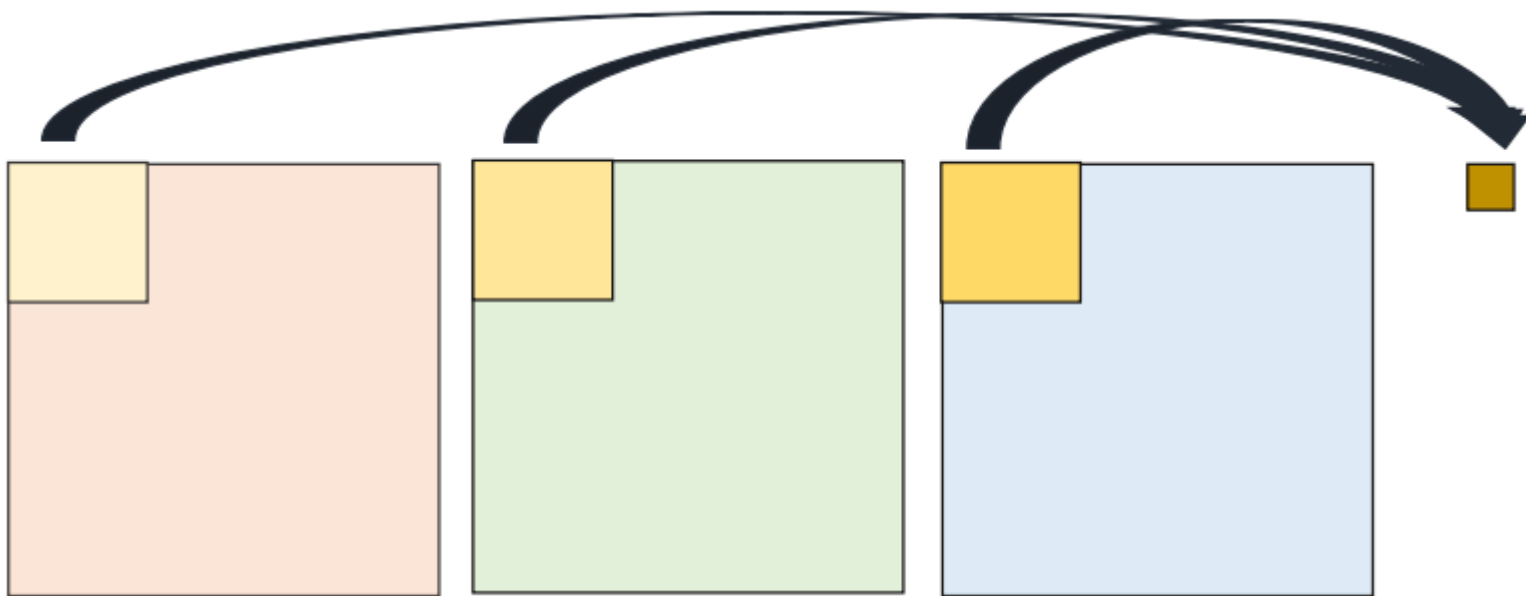
[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

Convolution Neural Networks (CNN)

Alexnet 구조 (Architecture of CNN)



[227x227x3] Input

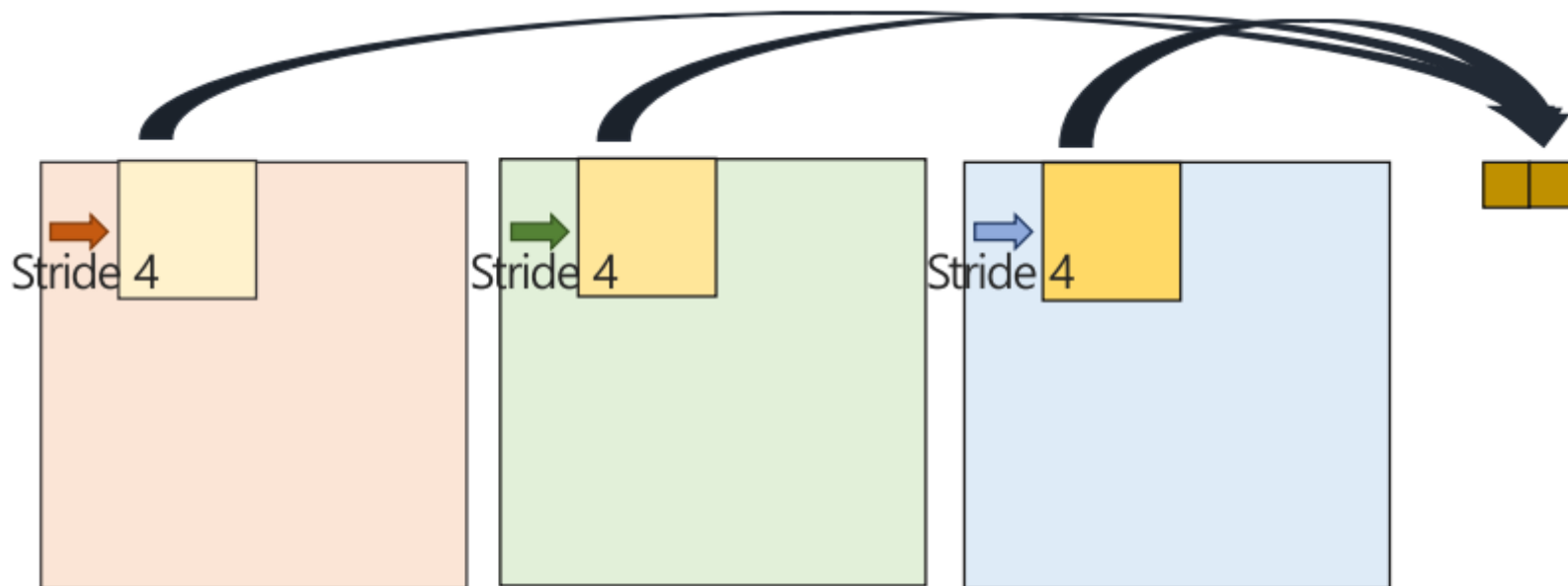
[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

Convolution Neural Networks (CNN)

Alexnet 구조 (Architecture of CNN)



[227x227x3] Input

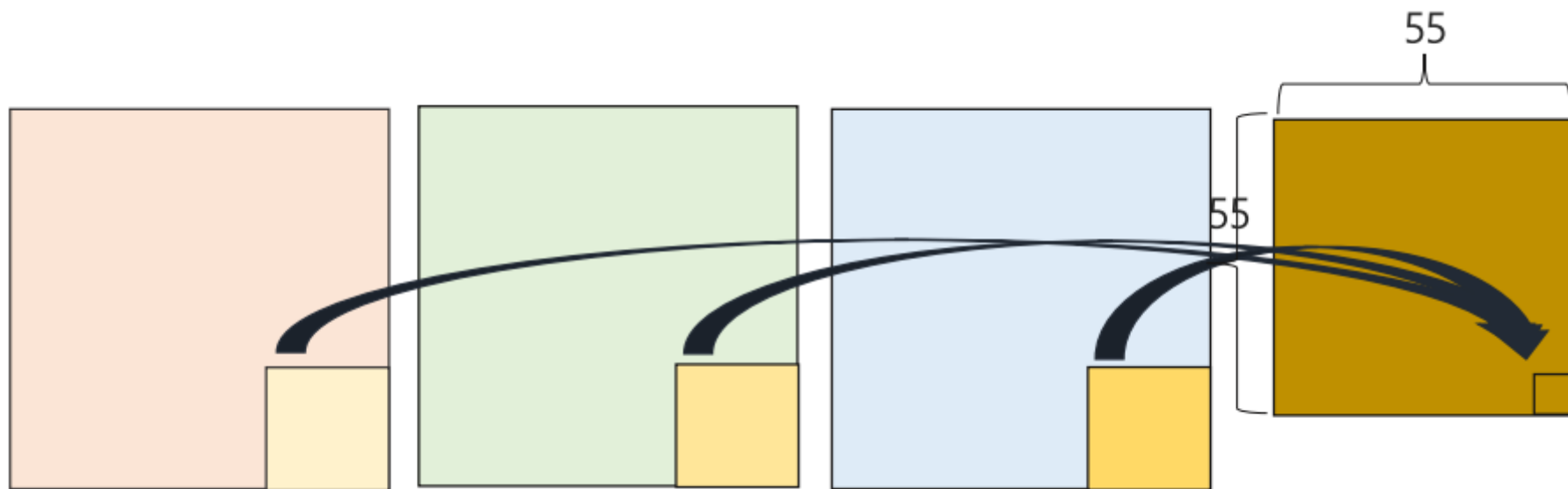
[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

Convolution Neural Networks (CNN)

Alexnet 구조 (Architecture of CNN)



[227x227x3] Input

[55x55x96] **Conv**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

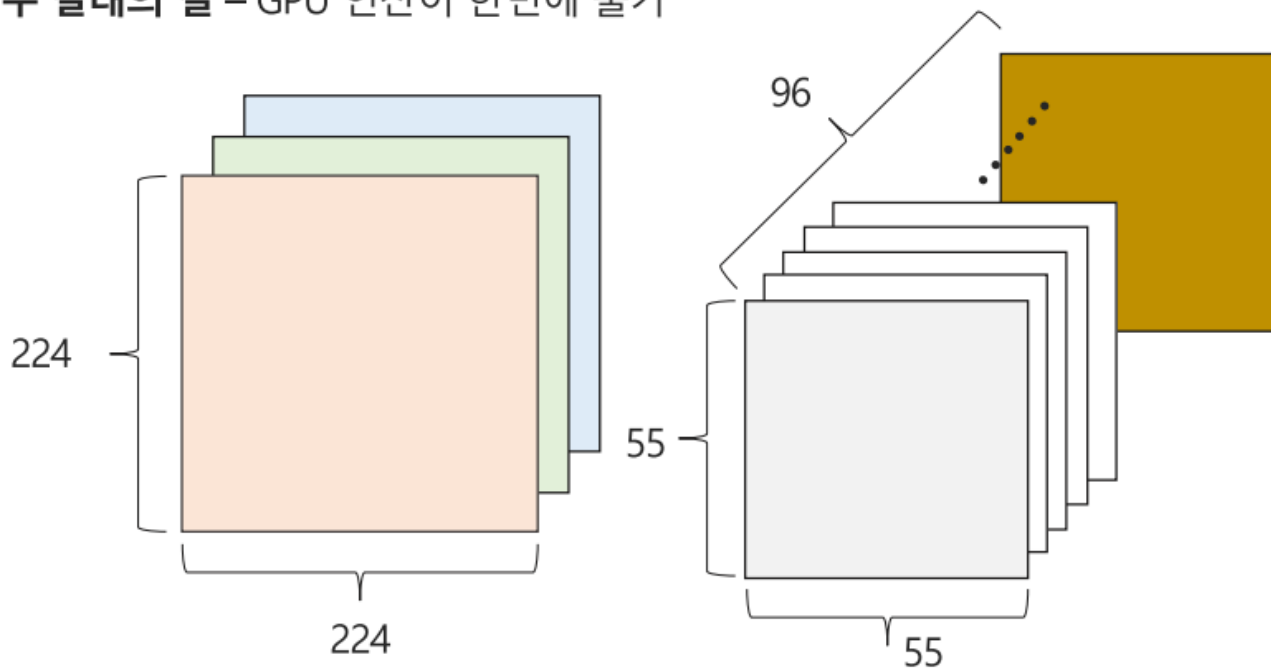
[27x27x96] **MAX POOL**: 3x3 filters at stride 2

[27x27x96] **NORM**: Normalization Layer

Convolution Neural Networks (CNN)

Alexnet 구조 (Architecture of CNN)

- ❖ 입력, 출력 $224 \times 224 \times 3$, 1000 class
- ❖ 파라미터 수 (60 million, 5 convolutional layers, 3 fully connected layers)
- ❖ 두 갈래의 길 – GPU 연산이 한번에 불가



[$227 \times 227 \times 3$] Input

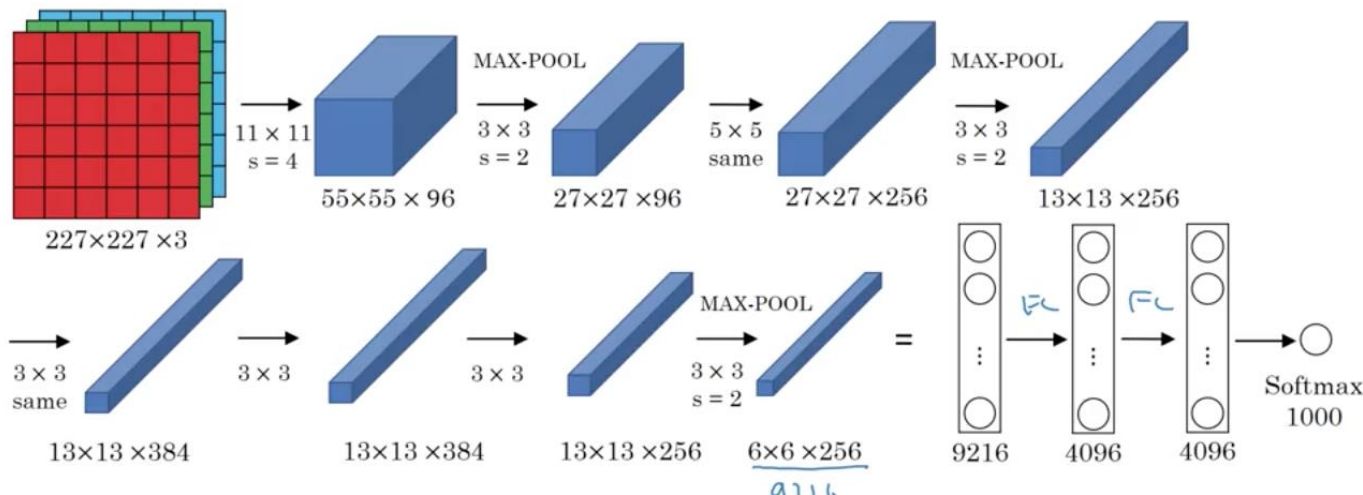
[$55 \times 55 \times 96$] **Conv1**: 96 11×11 filters at stride 4, pad 0 (96개의 다른 종류 11×11 filter 사용)

[$27 \times 27 \times 96$] **MAX POOL1**: 3×3 filters at stride 2

[$27 \times 27 \times 96$] **NORM1**: Normalization Layer

Convolution Neural Networks (CNN)

AlexNet



[227x227x3] Input

[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

[27x27x256] **Conv2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization Layer

[13x13x384] **Conv3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **Conv4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **Conv5**: 256 3x3 filters at stride 1, pad 1

[6 x 6 x 256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

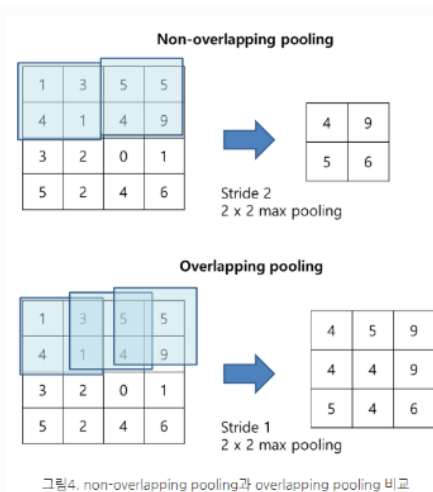


그림4. non-overlapping pooling과 overlapping pooling 비교

- **ReLU**
- **Multiple GPU**
- **Overlapping pooling**

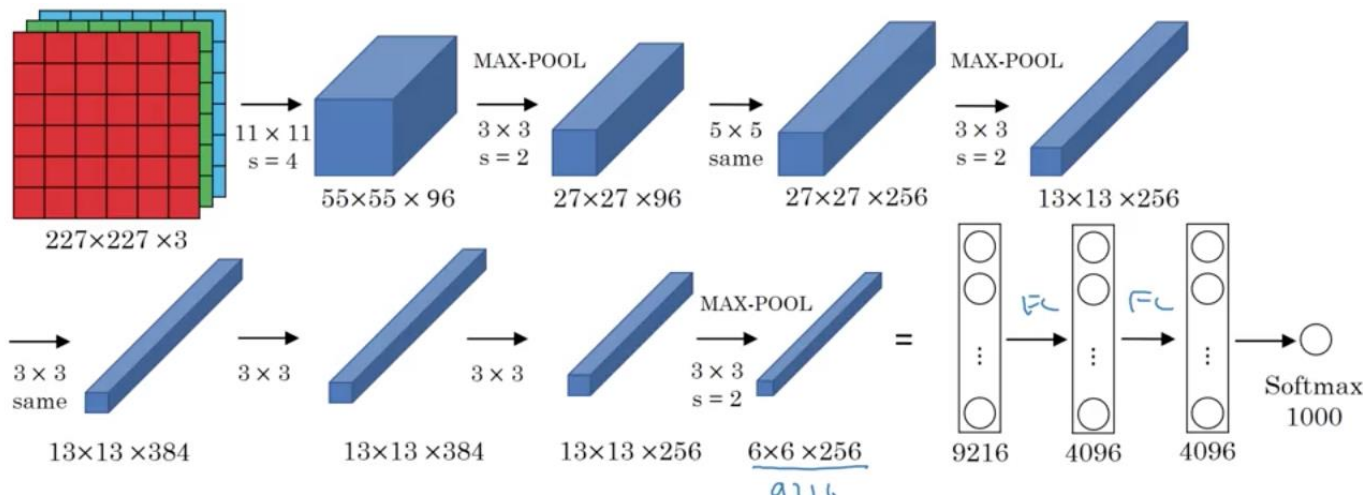
Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey. ImageNet clas

1 NIPS. 2012.

출처: bskyvision, [CNN 알고리즘들] AlexNet의 구조

Convolution Neural Networks (CNN)

AlexNet



[227x227x3] Input

[55x55x96] **Conv1**: 96 11x11 filters at stride 4, pad 0 (96개의 다른 종류 11x11 filter 사용)

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization Layer

[27x27x256] **Conv2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization Layer

[13x13x384] **Conv3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **Conv4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **Conv5**: 256 3x3 filters at stride 1, pad 1

[6 x 6 x 256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

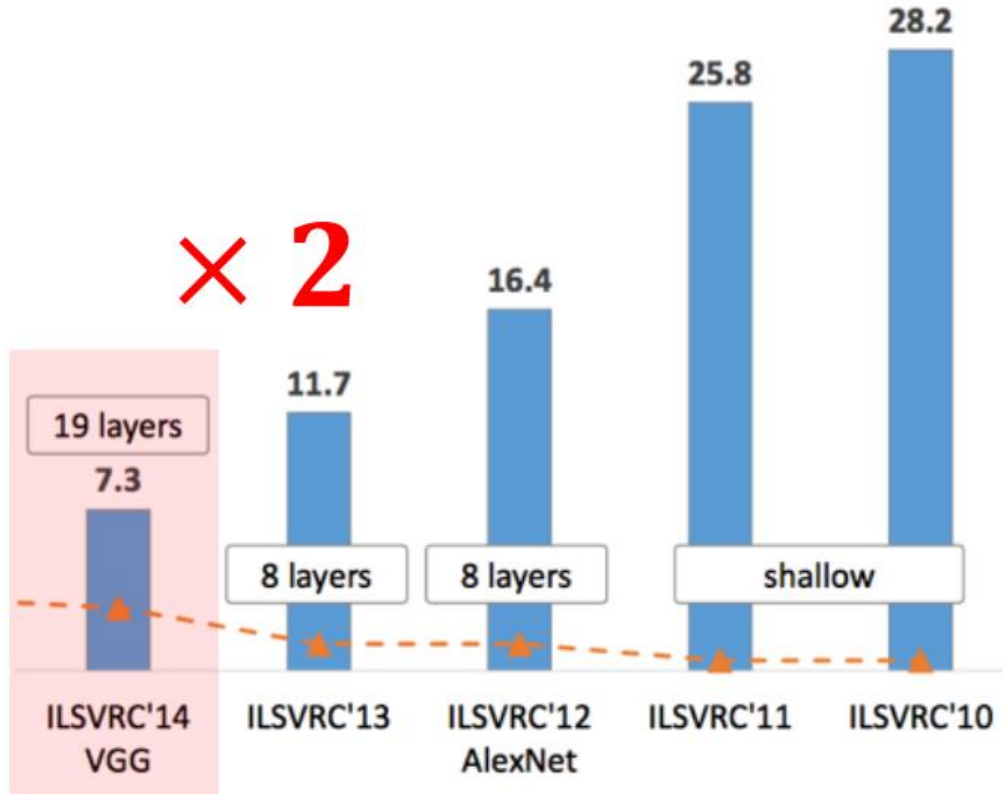
[1000] **FC8**: 1000 neurons (class scores)

- **ReLU**
- **Multiple GPU**
- **Overlapping pooling**
- **Dropout**

Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey. ImageNet classification with deep convolutional neural networks. In NIPS. 2012.

Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

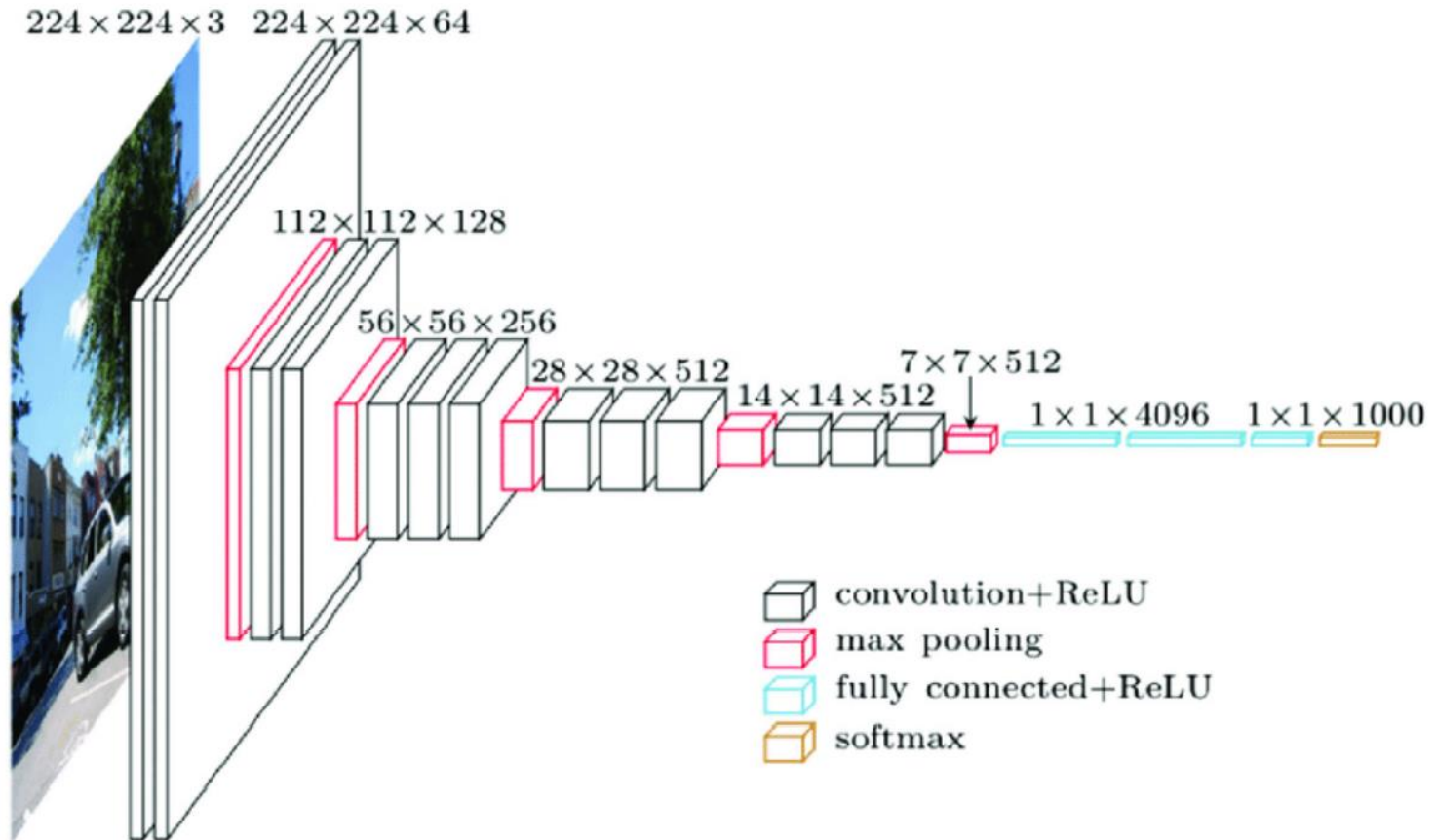


Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
<https://medium.com/@sidereal/cnns-architectures-lenet-alexnet-vgg-googlenet-resnet-and-more-666091488df5>

Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

❖ VGG16, VGG19



Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
https://www.researchgate.net/figure/An-overview-of-the-VGG-16-model-architecture-this-model-uses-simple-convolutional-blocks_fig2_328966158

Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- 224 × 224 픽셀 이미지

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

All Fully Connected Layer

224 × 224 image

FC-4096

FC-4096

FC-1000

226M 파라미터

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.

Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- 224 × 224 픽셀 이미지

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

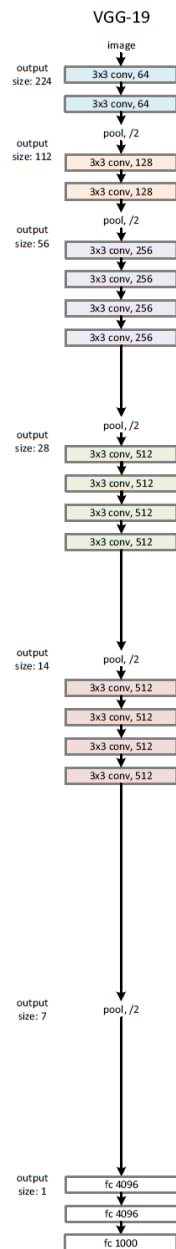
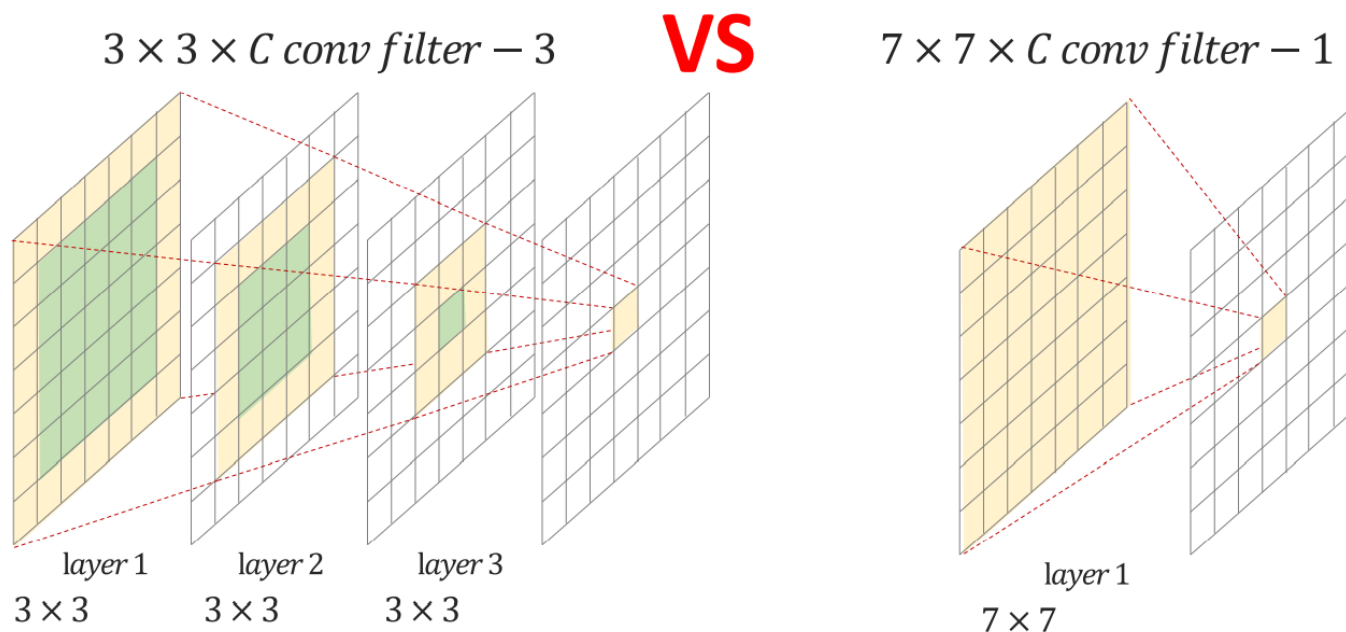
CNN 디자인의 가이드 제시

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.

Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

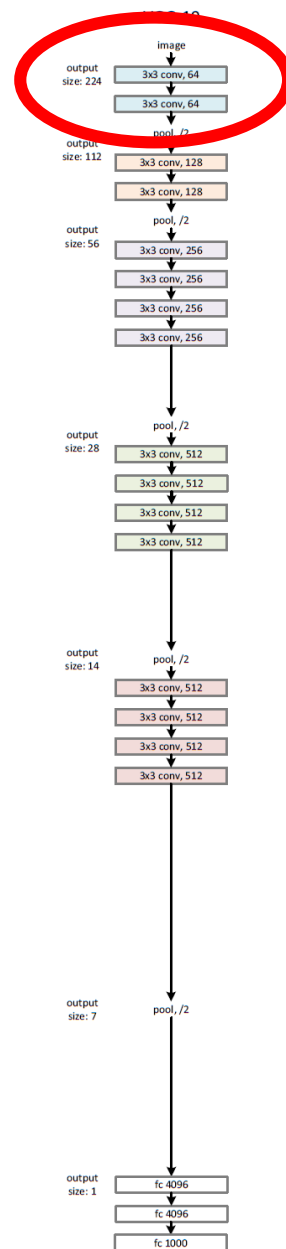
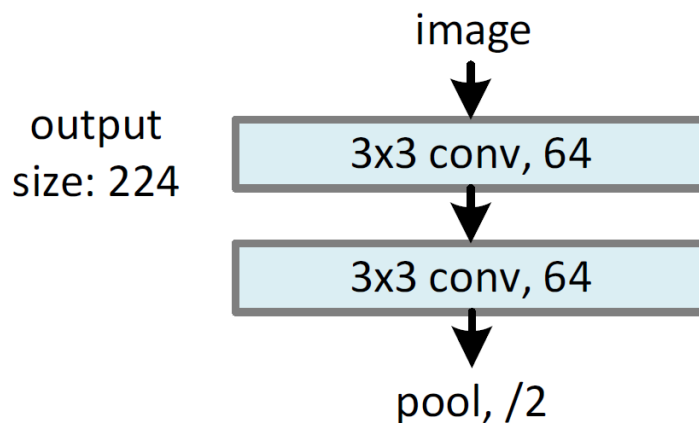
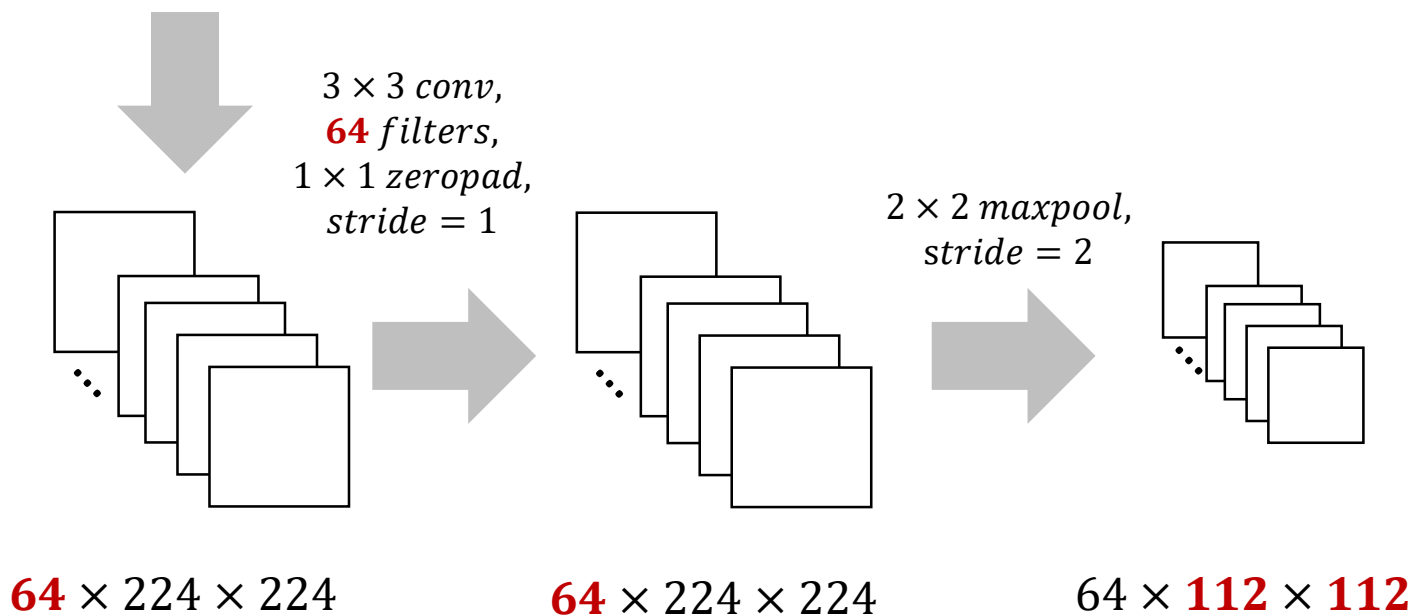
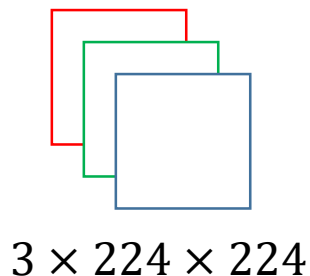
- 224×224 픽셀 이미지 분류
- 2nd place in ILSVRC 2014
- **3×3 필터를 사용!**
- Image size $/2 \rightarrow$ # filters $\times 2$!



Convolution Neural Networks (CNN)

VGGNet

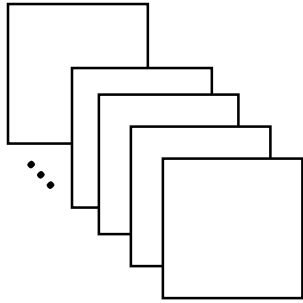
Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford



Convolution Neural Networks (CNN)

VGGNet

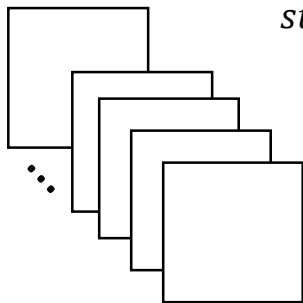
Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford



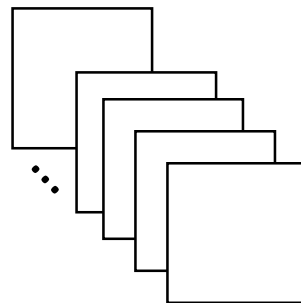
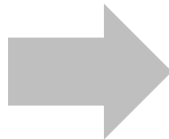
$64 \times 112 \times 112$



3×3 conv,
128 filters,
 1×1 zeropad,
stride = 1

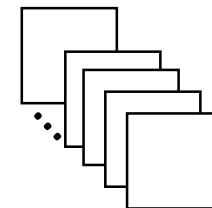


$128 \times 112 \times 112$



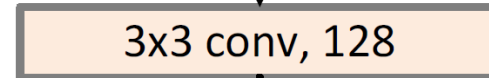
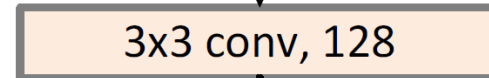
$128 \times 112 \times 112$

2×2 maxpool,
stride = 2

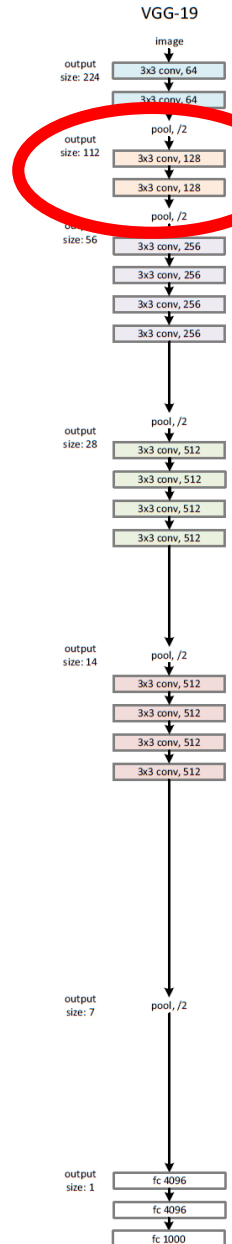


$128 \times 56 \times 56$

output
size: 112



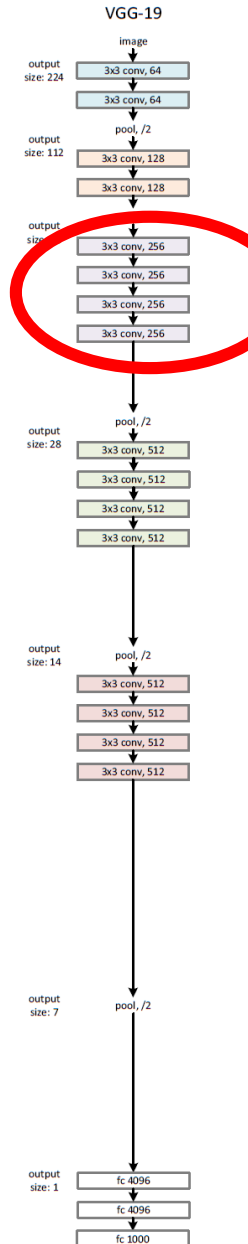
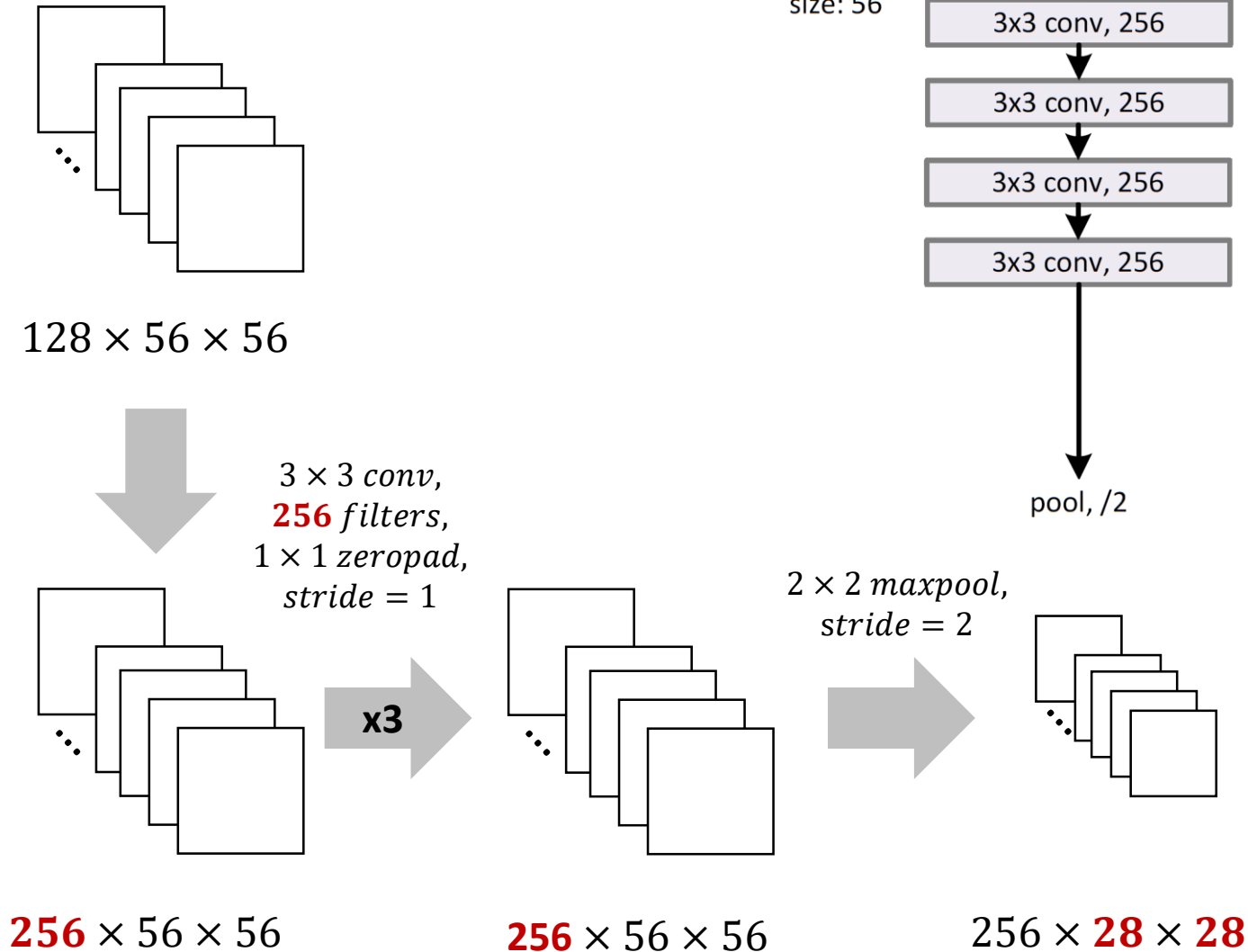
pool, /2



Convolution Neural Networks (CNN)

VGGNet

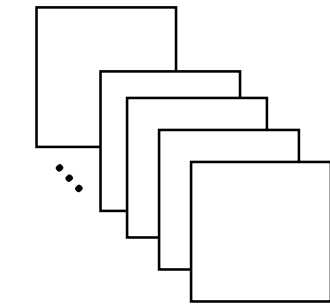
Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford



Convolution Neural Networks (CNN)

VGGNet

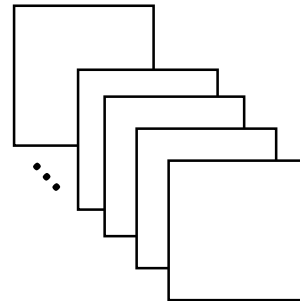
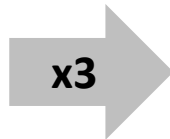
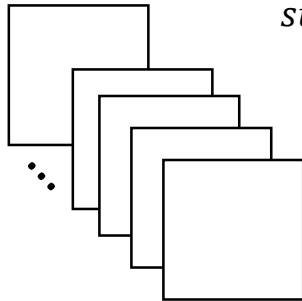
Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford



$256 \times 28 \times 28$



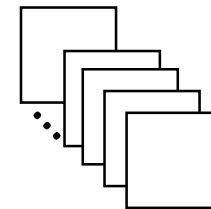
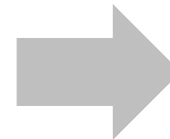
3×3 conv,
512 filters,
 1×1 zeropad,
stride = 1



$512 \times 28 \times 28$

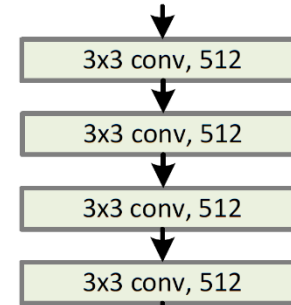
$512 \times 28 \times 28$

2×2 maxpool,
stride = 2

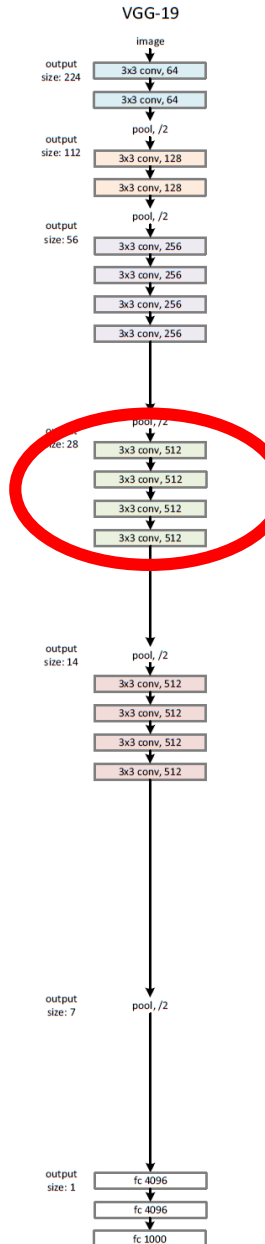


$512 \times 14 \times 14$

output
size: 28



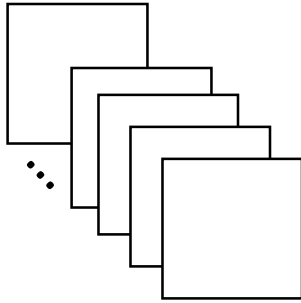
pool, /2



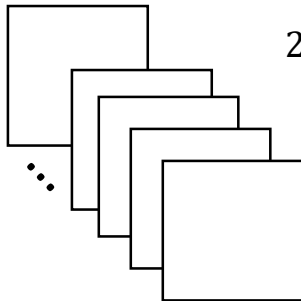
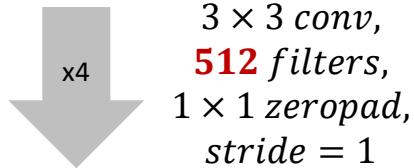
Convolution Neural Networks (CNN)

VGGNet

Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

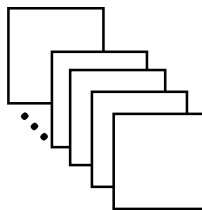


$512 \times 14 \times 14$

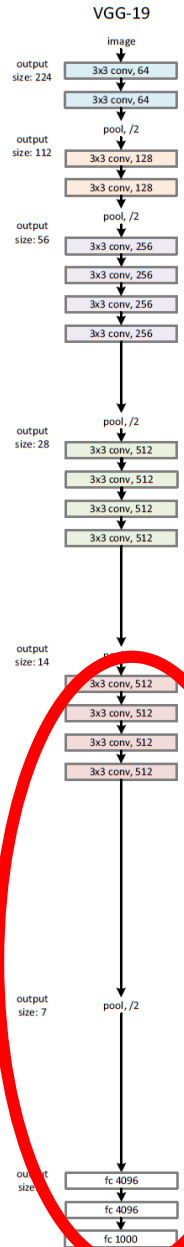
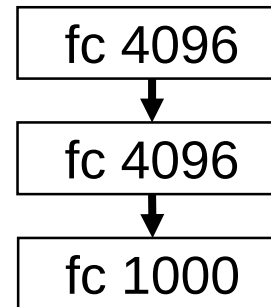


$512 \times 14 \times 14$

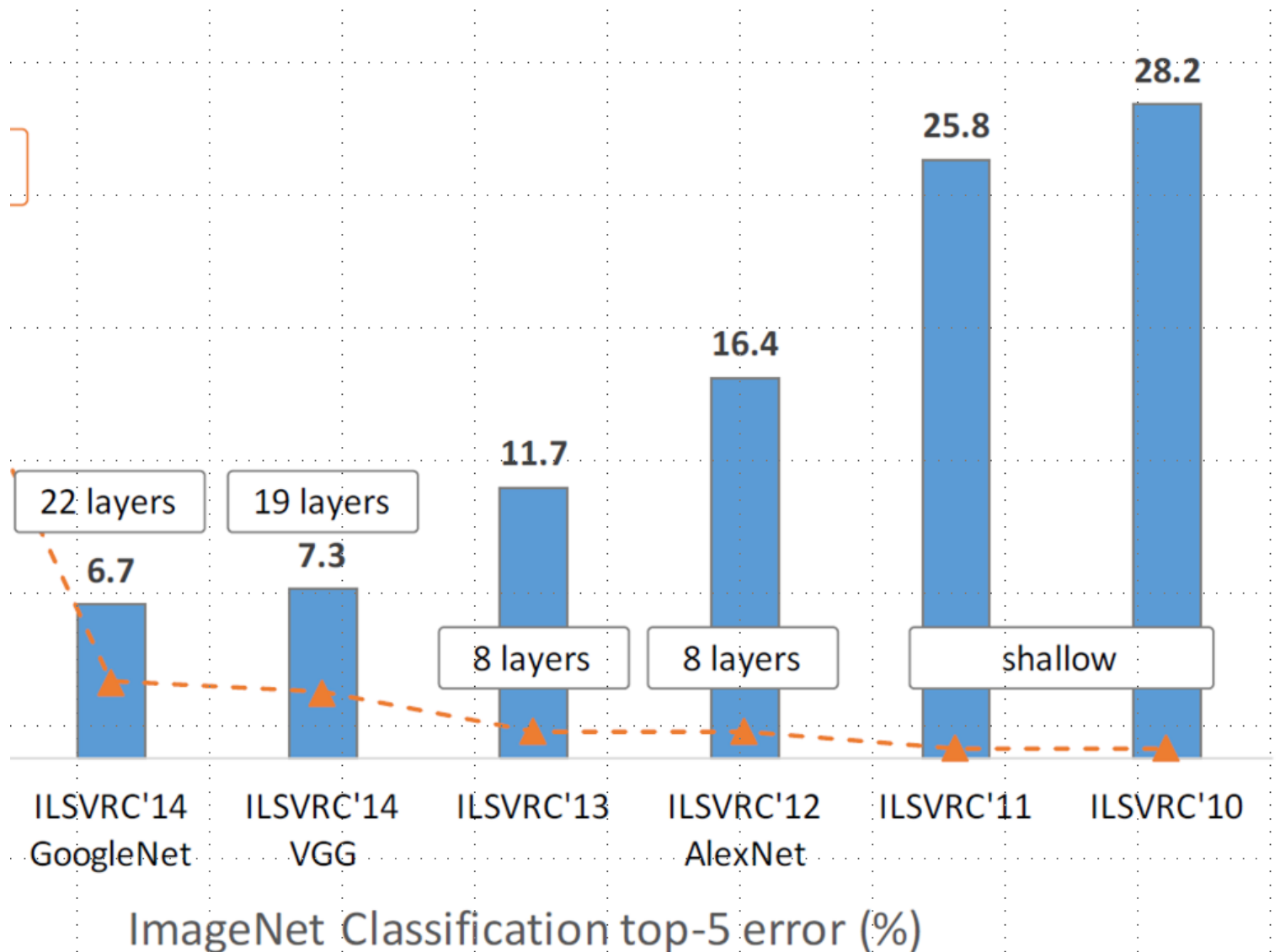
$2 \times 2 \text{ maxpool,}$
 $\text{stride} = 2$



$512 \times \mathbf{7} \times \mathbf{7}$

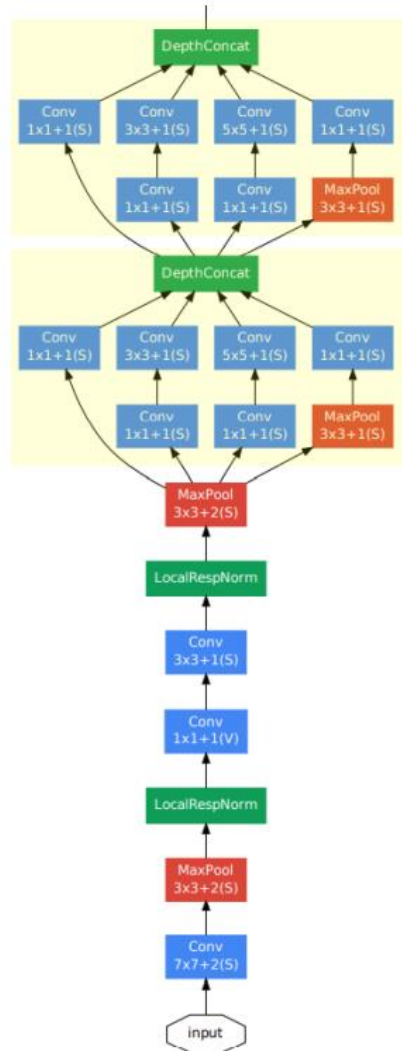


Convolution Neural Networks (CNN)



Convolution Neural Networks (CNN)

GoogLeNet (Inception)



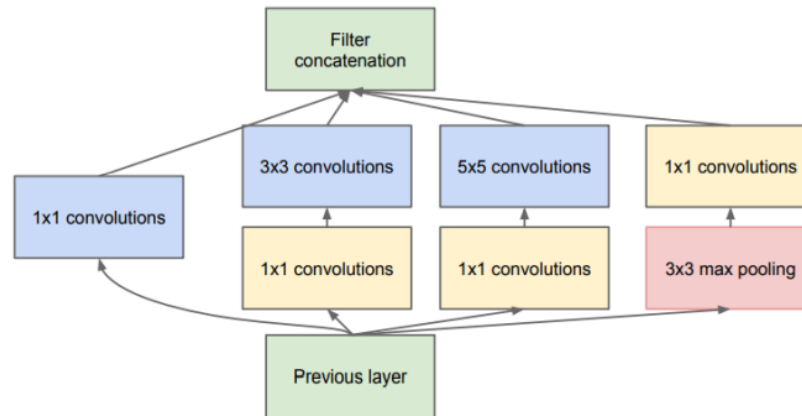
Convolution Neural Networks (CNN)

GoogLeNet (Inception)

❖ Inception 모듈

- 모델 복잡도 (complexity)는 유지하면서 모델을 더 깊게 (depth) , 더 넓게 (width)
- one by one convolution = dimension (channel) reduction
- 갈림길 장점, receptive field 고정 X, 다양화

■ 앙상블 기법 중 하나



Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., ... & Rabinovich, A. (2015). Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1-9).

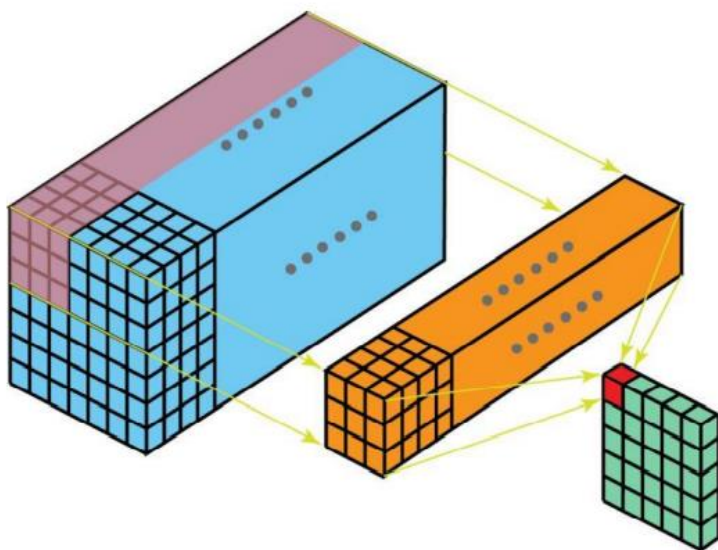
Convolution Neural Networks (CNN)

GoogLeNet (Inception)

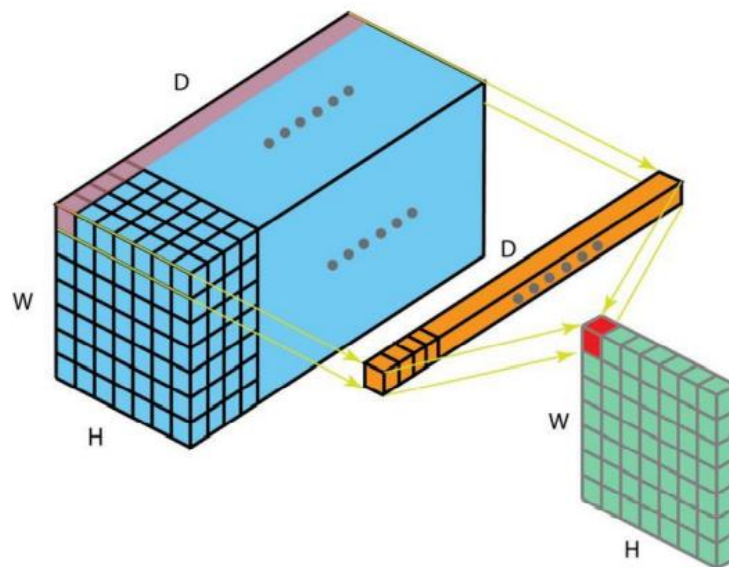
❖ Inception 모듈

- 모델 복잡도 (complexity)는 유지하면서 모델을 더 깊게 (depth), 더 넓게 (width)
- **one by one convolution = dimension (channel) reduction**
- 갈림길 장점, receptive field 고정 X, 다양화

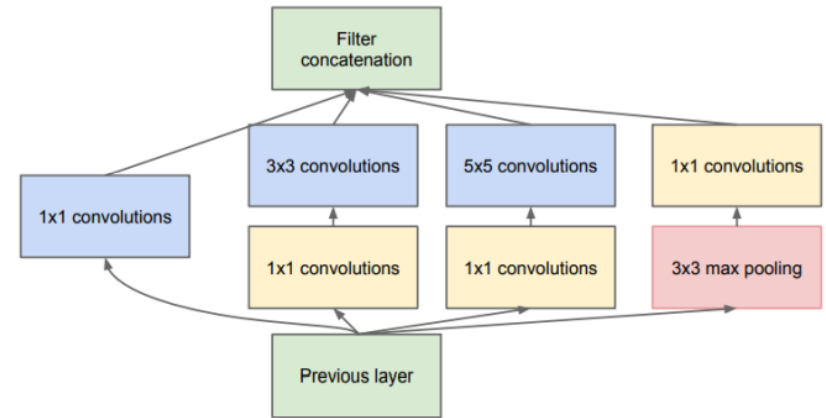
3×3 convolution



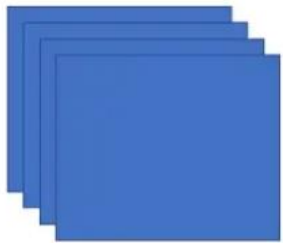
1×1 convolution



Convolution Neural Networks (CNN)



Stack of 64 feature maps



32x32x64

Convolution
1x1x16

Convolution
3x3x32
Padding = same,
stride = 1

Max pooling
3x3, padding =
same, stride = 1



32x32x16



32x32x32



32x32x64



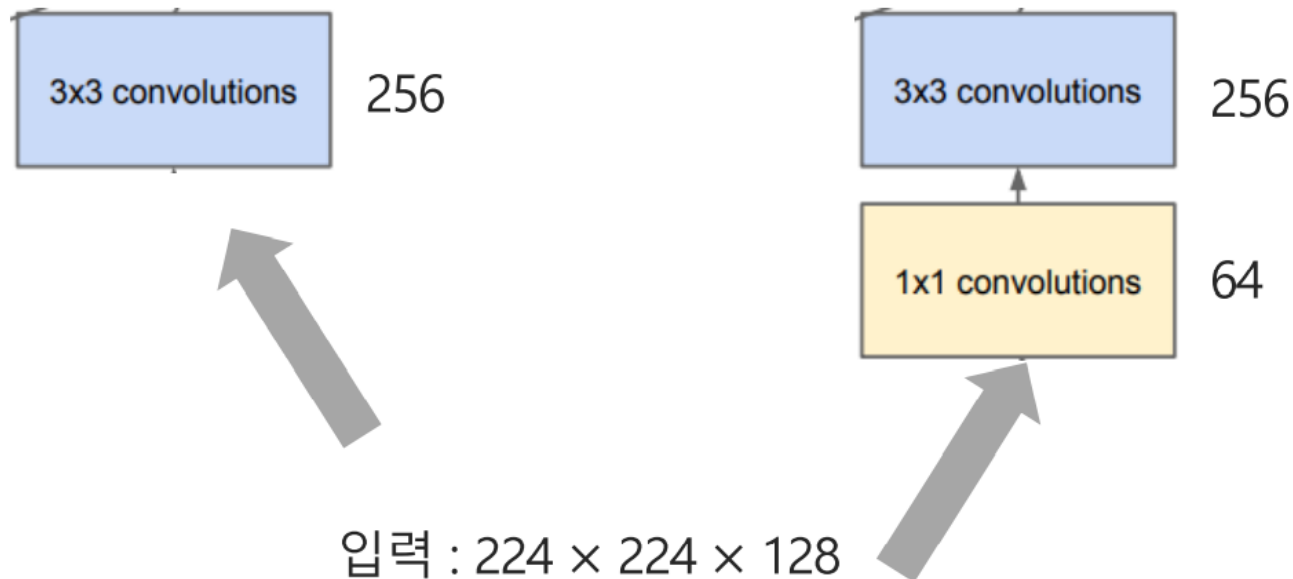
32x32x112 (112 = 16+32+64)

Convolution Neural Networks (CNN)

GoogLeNet (Inception)

❖ Inception 모듈

- 복잡도 (complexity)는 유지하면서 모델을 더 깊게 (depth), 더 넓게 (width)
- **one by one convolution = dimension (channel) reduction**
- 갈림길 장점, receptive field 고정 X, 다양화

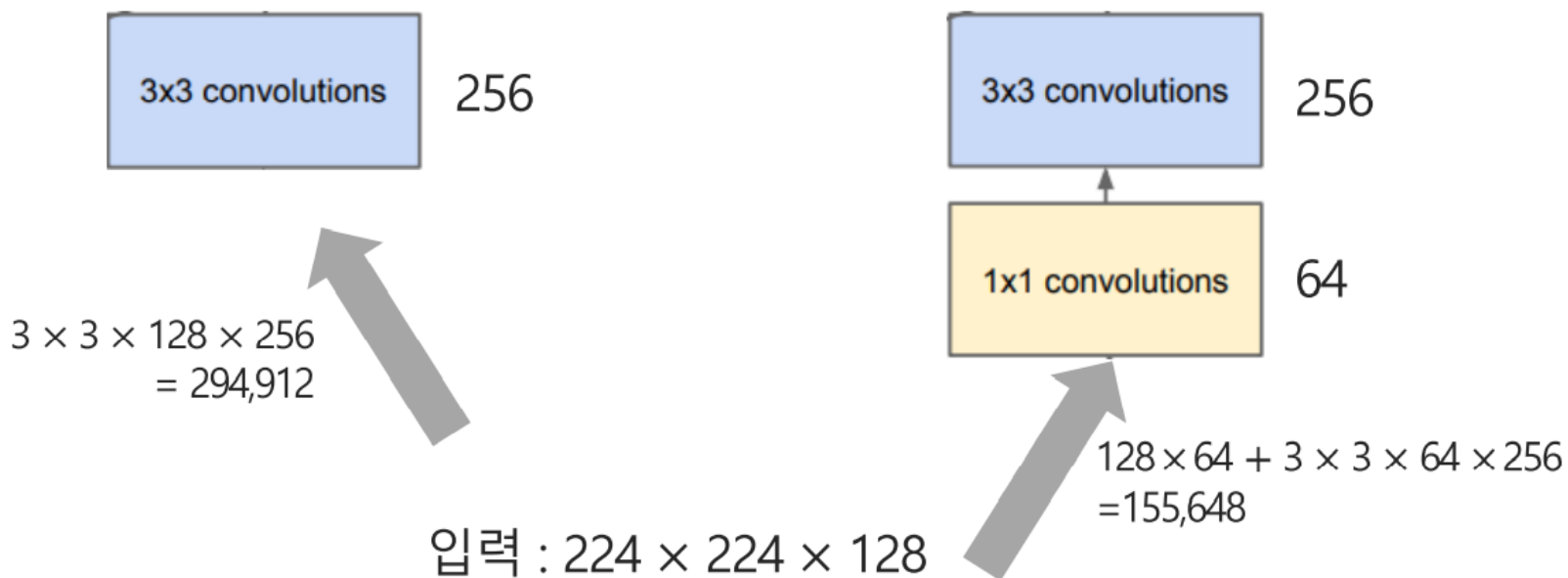


Convolution Neural Networks (CNN)

GoogLeNet (Inception)

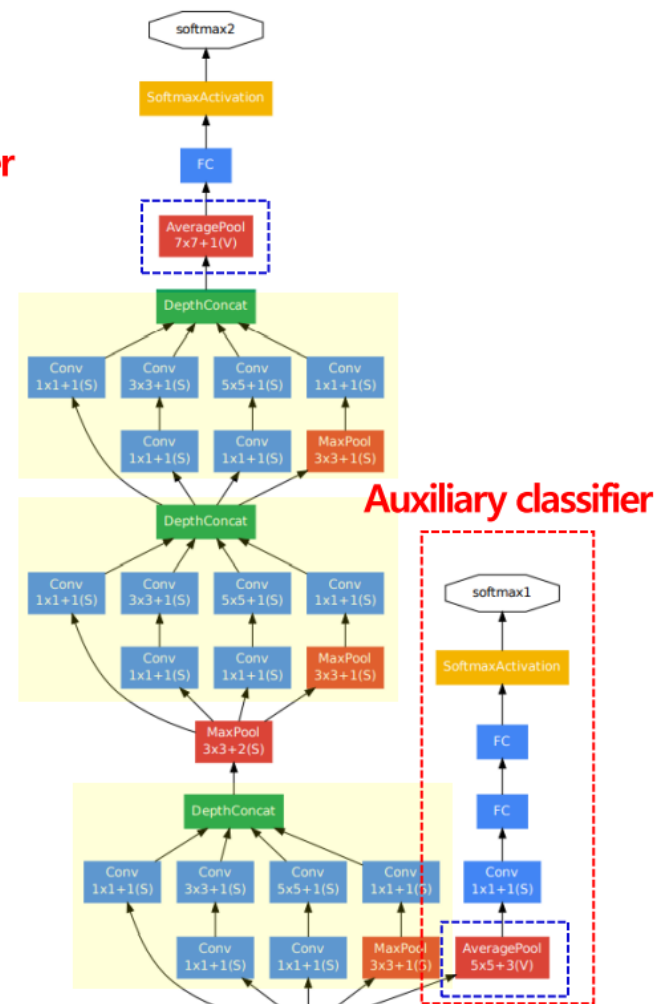
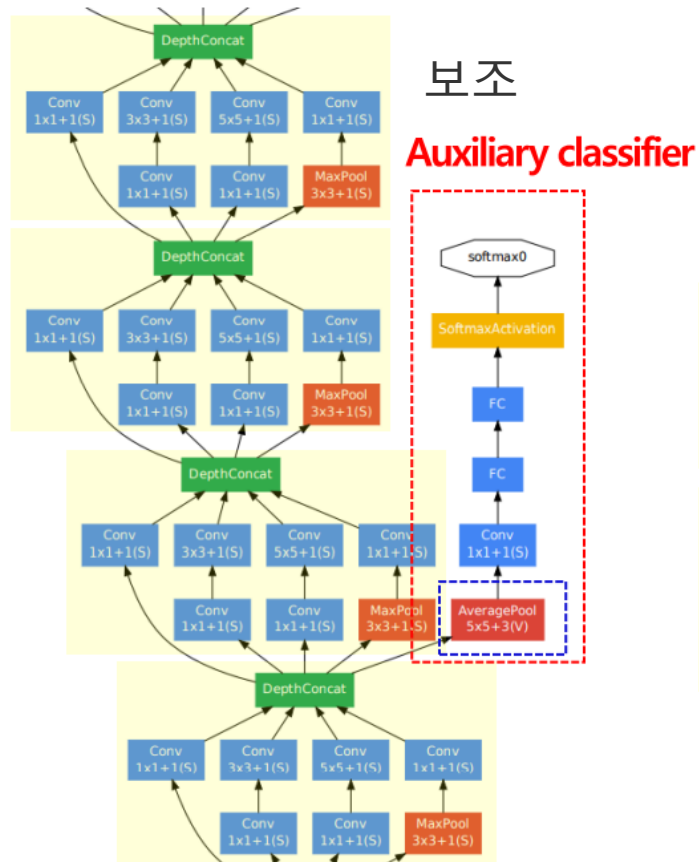
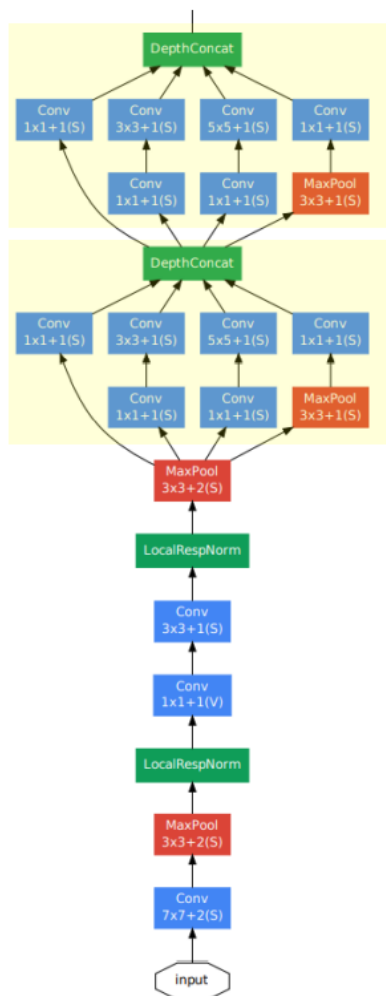
❖ Inception 모듈

- 복잡도 (complexity)는 유지하면서 모델을 더 깊게 (depth), 더 넓게 (width)
- **one by one convolution = dimension (channel) reduction**
- 갈림길 장점, receptive field 고정 X, 다양화



Convolution Neural Networks (CNN)

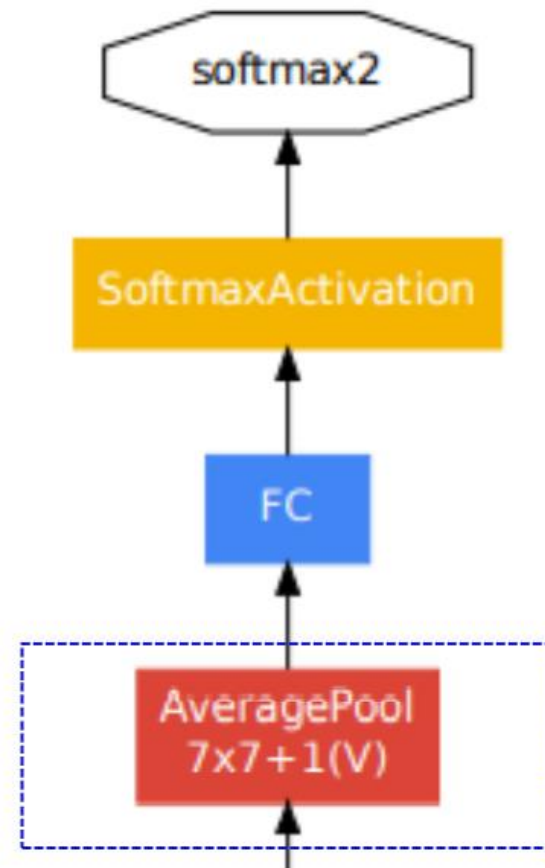
GoogLeNet (Inception)



Convolution Neural Networks (CNN)

GoogLeNet (Inception)

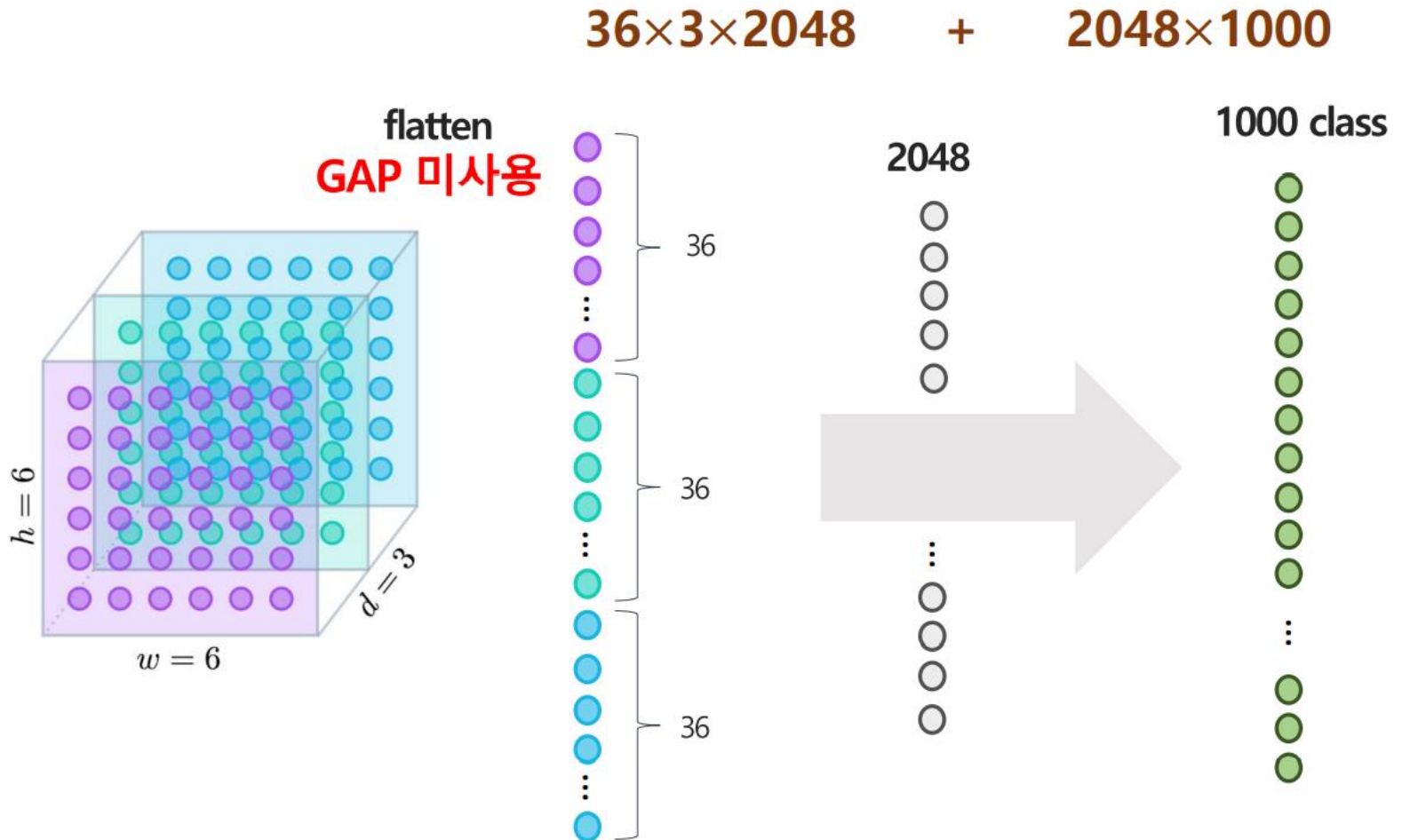
❖ Global Average Pooling



Convolution Neural Networks (CNN)

GoogLeNet (Inception)

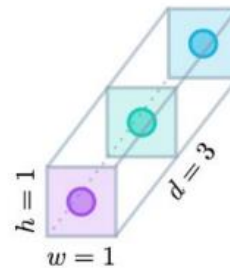
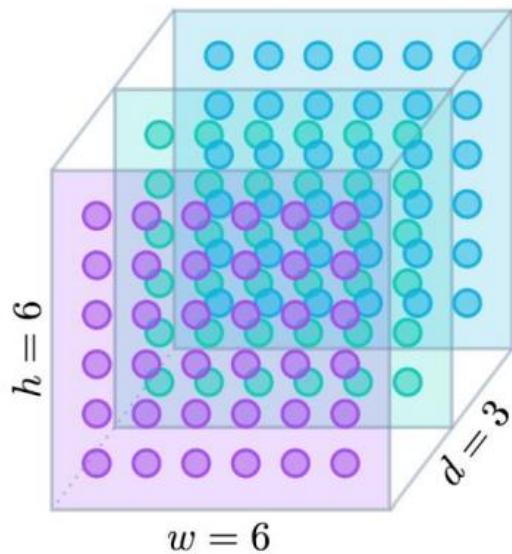
❖ Global Average Pooling



Convolution Neural Networks (CNN)

GoogLeNet (Inception)

❖ Global Average Pooling



3×2048

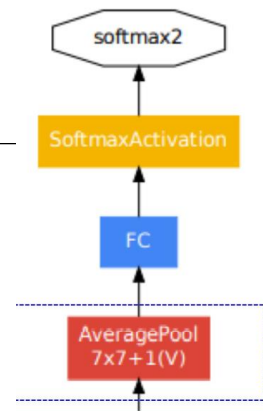
+

2048×1000

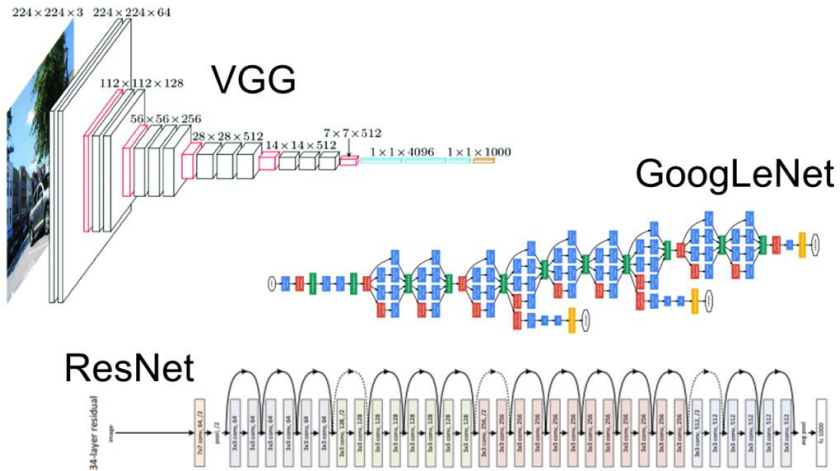
2048



1000 class

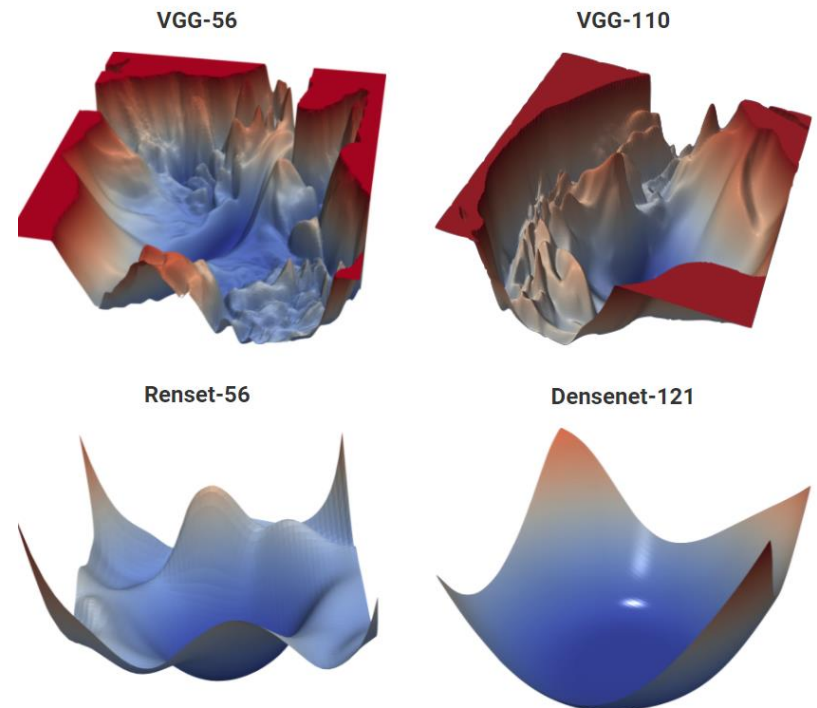


Deep Learning



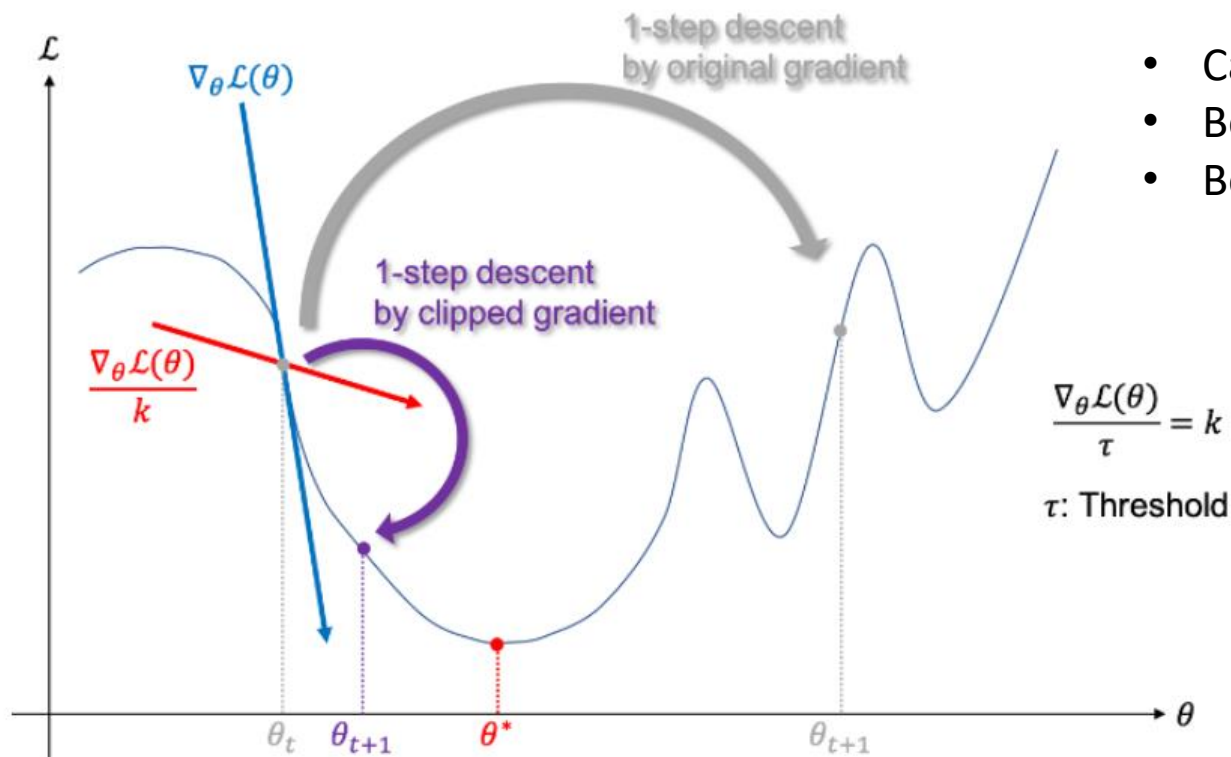
ResNet 50, 152: # of layers = 50, 152

Parameter Space



Deep Neural Network (DNN, 심층 신경망)

Vanishing Gradient vs Exploding Gradients



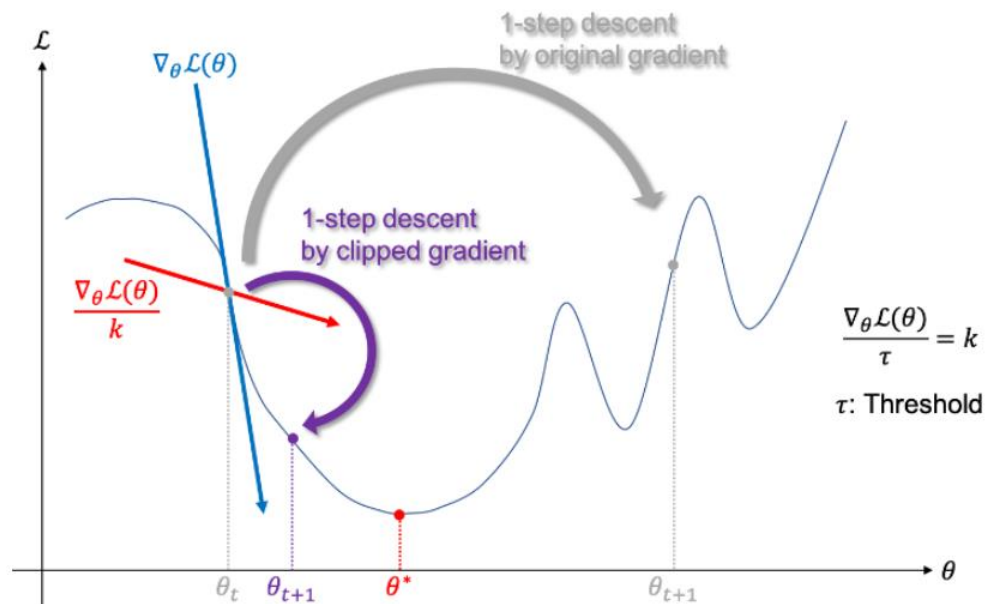
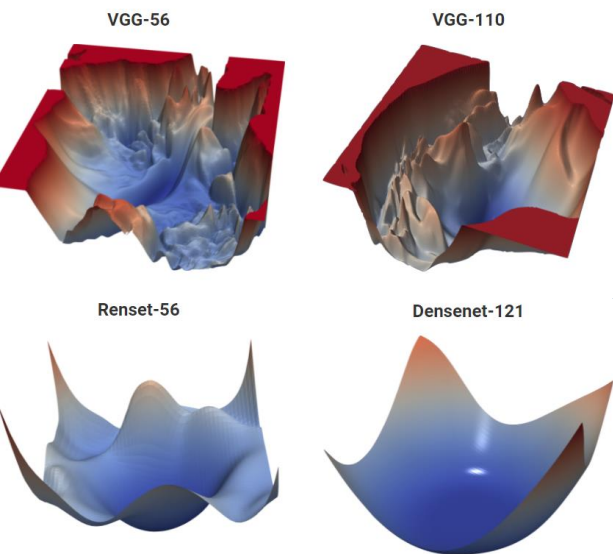
https://velog.io/@nayeon_p00/%EB%94%A5%EB%9F%AC%EB%8B%9D-%EA%B8%B0%EC%B4%88%EA%B0%9C%EB%85%90-%EA%B8%B0%EC%9A%B8%EA%B8%B0-%EC%86%8C%EC%8B%A4%EA%B3%BC-%ED%8F%AD%EC%A3%BC-Gradient-Vanishing-Exploding

- Carefully setting Learning rate
- Better Optimizer
- Better Activation function

Deep Neural Network (DNN, 심층 신경망)

모든 문제는 NN의 복잡성 때문

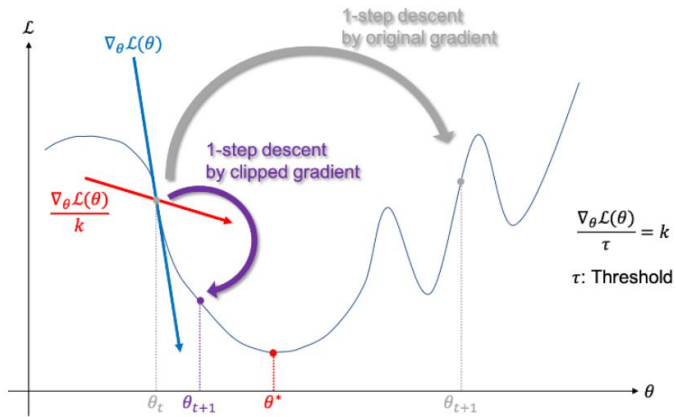
Parameter Space



https://velog.io/@nayeon_p00/%EB%94%A5%EB%9F%AC%EB%8B%9D-%EA%B8%B0%EC%B4%88%EA%B0%9C%EB%85%90-%EA%B8%B0%EC%9A%B8%BA-%EC%86%8C%EC%8B%A4%EA%B3%BC-%ED%8F%AD%EC%A3%BC-Gradient-Vanishing-Exploding

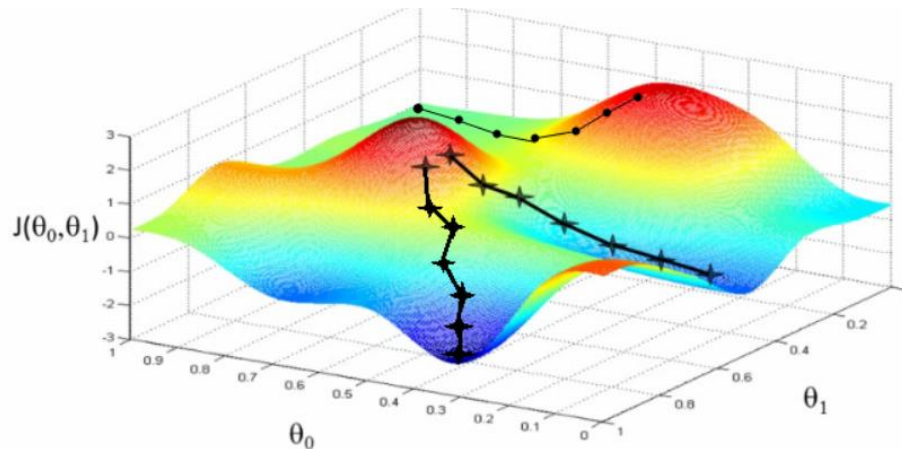
Deep Neural Network (DNN, 심층 신경망)

Vanishing Gradient vs Exploding Gradients



https://velog.io/@nayeon_p00/%EB%94%A5%EB%9F%AC%EB%8B%9D-%EA%B8%B0%EC%B4%88%EA%B0%9C%EB%85%90-%EA%B8%B0%EC%9A%B8%EA%B8%B0-%EC%86%8C%EC%8B%A4%EA%B3%BC-%ED%8F%AD%EC%A3%BC-Gradient-Vanishing-Exploding

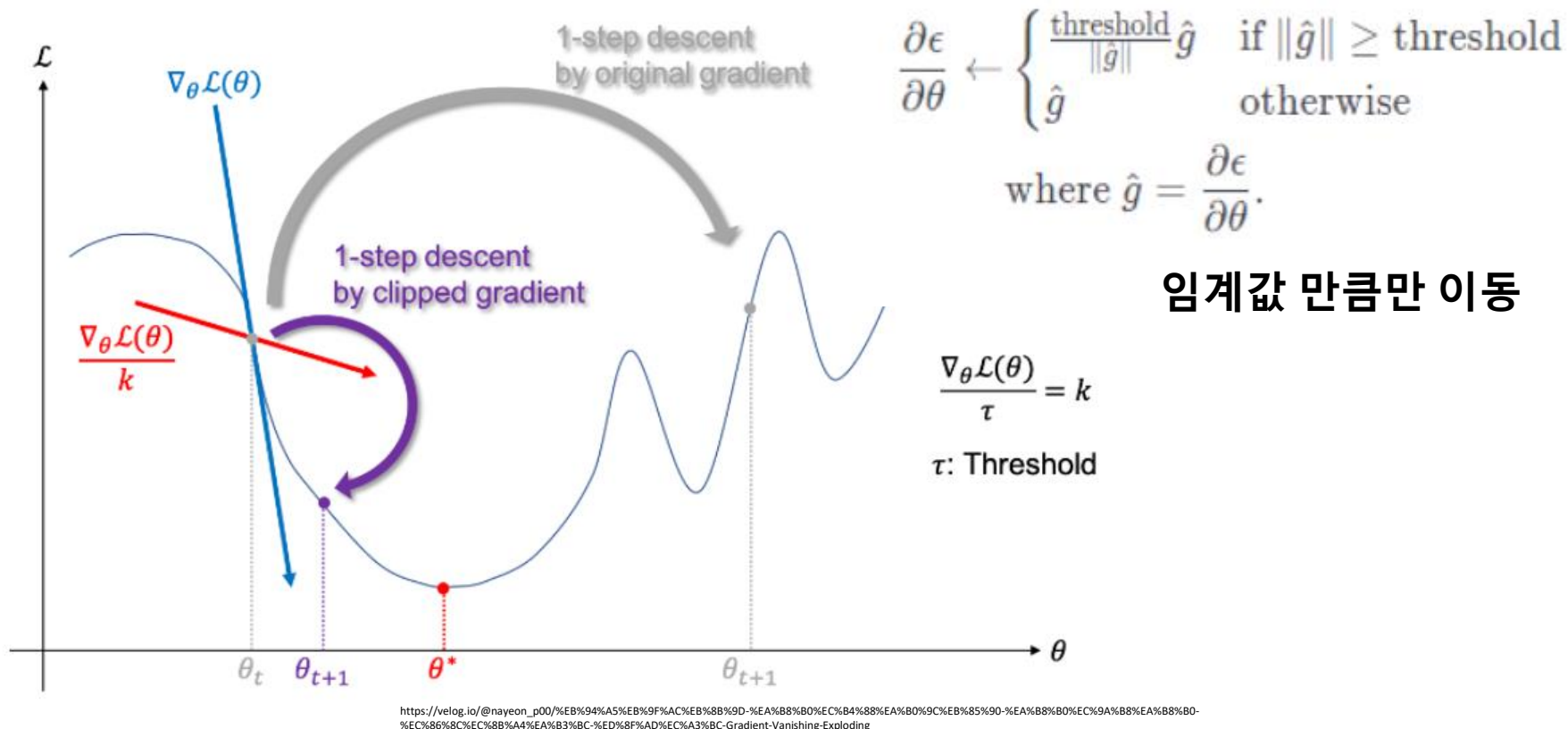
- Carefully setting Learning rate
- Better Activation function
- **Better Optimizer**
- Careful initialization of weights
- Turning data distribution – Batch Normalization



Deep Neural Network (DNN, 심층 신경망)

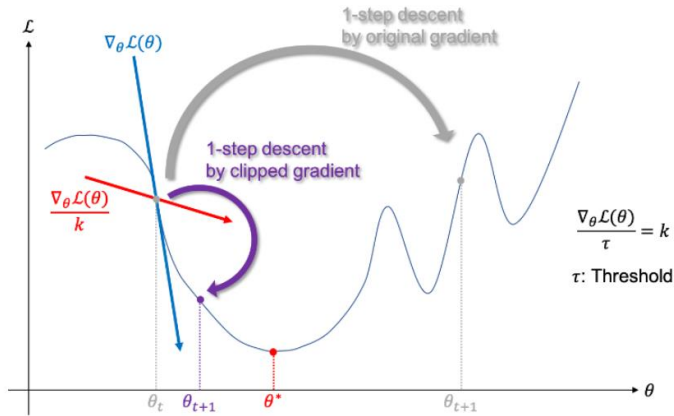
Vanishing Gradient vs Exploding Gradients

Gradient Clipping



Deep Neural Network (DNN, 심층 신경망)

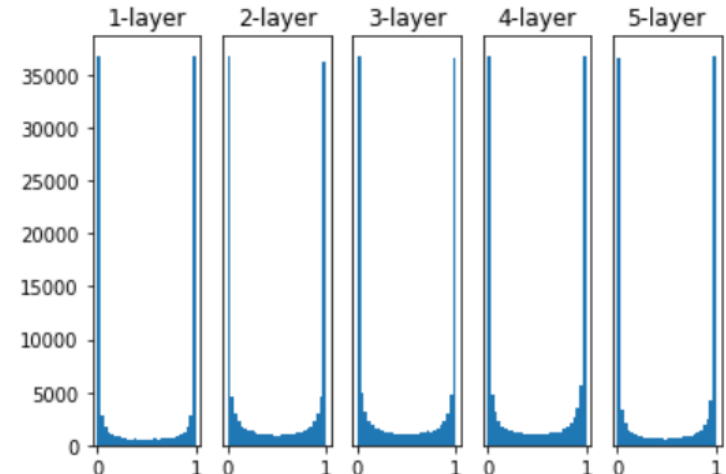
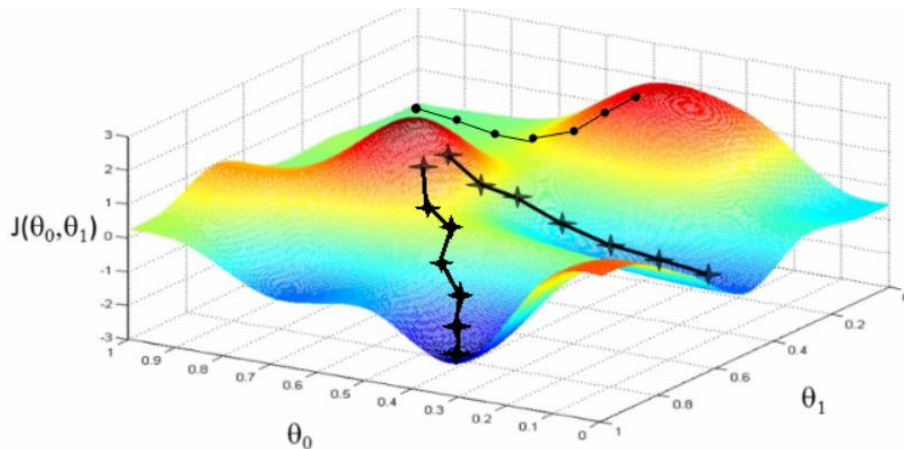
Vanishing Gradient vs Exploding Gradients



https://velog.io/@nayeon_p00/%EB%94%A5%EB%9F%AC%EB%8B%9D-%EA%B8%B0%EC%B4%88%EA%B0%9C%EB%85%90-%EA%B8%B0%EC%9A%B8%EA%B8%B0-%EC%86%8C%EC%8B%A4%EA%B3%BC-%ED%8F%AD%EC%A3%BC-Gradient-Vanishing-Exploding

- Carefully setting Learning rate
- Better Optimizer
- Better Activation function
- Careful initialization of weights
- Turning data distribution – Batch Normalization

표준편차가 1인



Deep Neural Network (DNN, 심층 신경망)

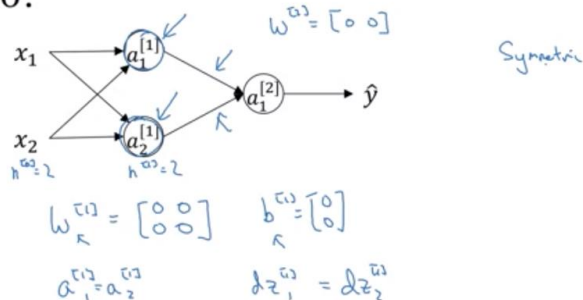
Vanishing Gradient vs Exploding Gradients

모든 문제는 NN의 복잡성 때문

초기 가중치 설정 (weight initialization)

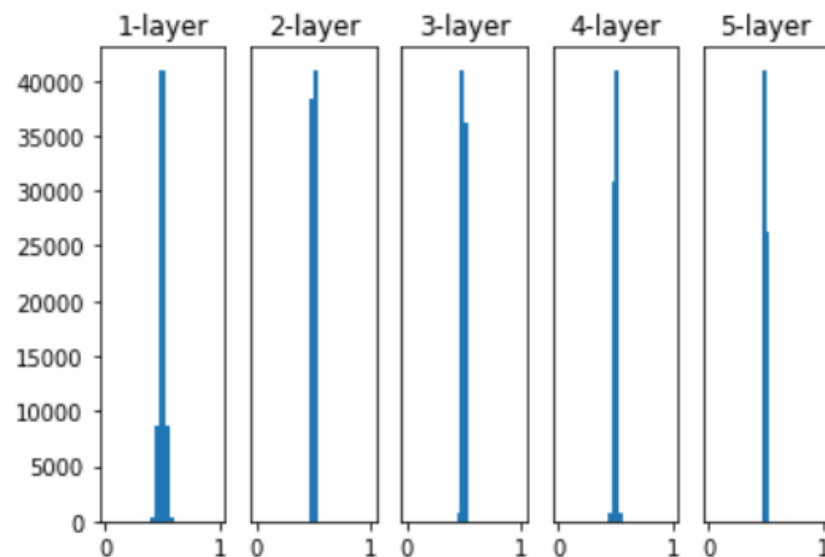
Random Initialization

What happens if you initialize weights to zero?



파라미터의 값이 모두 같다면 [역전파\(Back propagation\)](#)를 통해서 갱신하더라도 모두 같은 값으로 변하게 됩니다.
신경망 노드의 파라미터가 모두 동일하다면 여러 개의 노드로 신경망을 구성하는 의미가 사라집니다.

표준편차가 0.5인



Deep Neural Network (DNN, 심층 신경망)

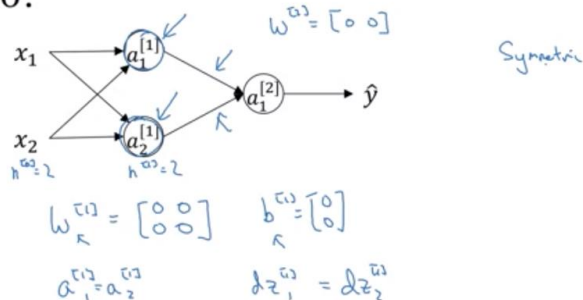
Vanishing Gradient vs Exploding Gradients

모든 문제는 NN의 복잡성 때문

초기 가중치 설정 (weight initialization) - Xavier Initialization

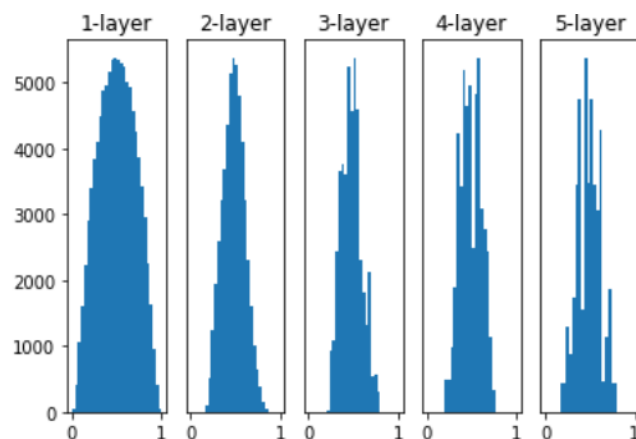
Random Initialization

What happens if you initialize weights to zero?

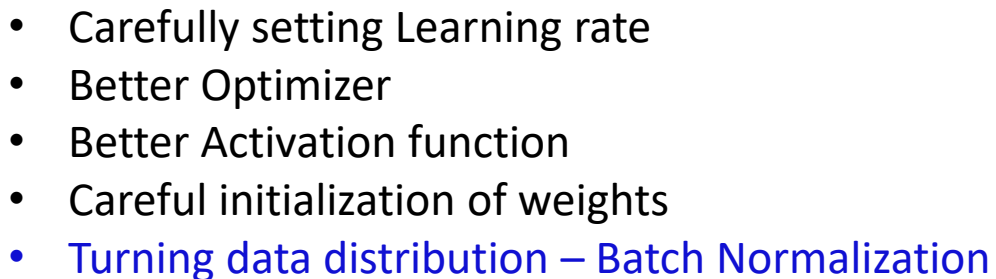


파라미터의 값이 모두 같다면 [역전파\(Back propagation\)](#)를 통해서 갱신하더라도 모두 같은 값으로 변하게됩니다.
신경망 노드의 파라미터가 모두 동일하다면 여러 개의 노드로 신경망을 구성하는 의미가 사라집니다.

표준편차가 $\frac{2}{\sqrt{n+m}}$ n =이전 레이어노드 수
 m =현재 레이어노드수



Vanishing Gradient vs Exploding Gradients

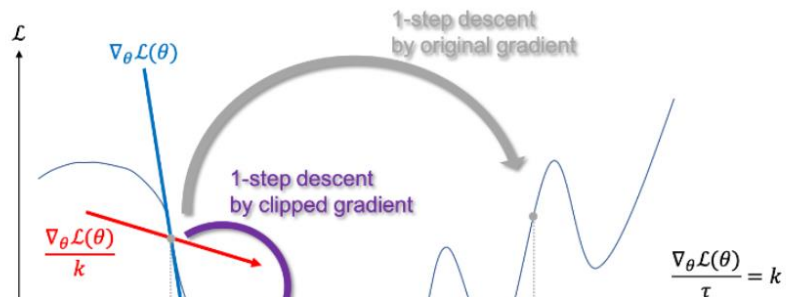


Internal Covariate Shift



Deep Neural Network (DNN, 심층 신경망)

Vanishing Gradient vs Exploding Gradients



- Carefully setting Learning rate
- Better Optimizer
- Better Activation function
- Careful initialization of weights
- Turning data distribution – Batch Normalization

Subtract the Mean

Calculate batch mean: $\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$

Subtract the mean: $\hat{x}_i \leftarrow x_i - \mu_B$



Normalize Variance

Calculate Variance: $\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$

Subtract the mean, divide by standard deviation: $\tilde{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$



Image created by author.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad // \text{ mini-batch variance}$$

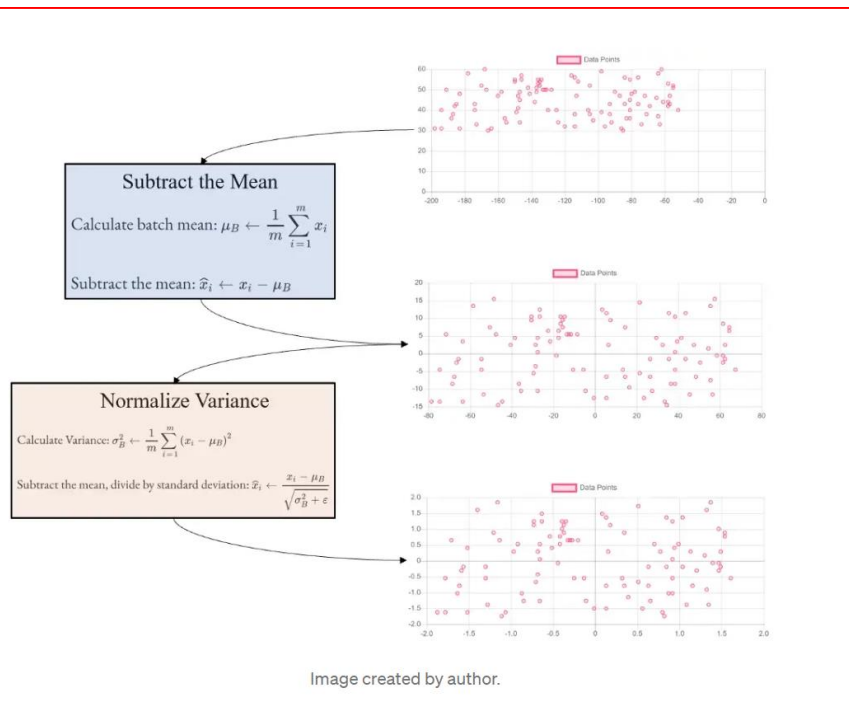
$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Deep Neural Network (DNN, 심층 신경망)

Vanishing Gradient vs Exploding Gradients

“Batch normalization



- *More freedom in setting the initial learning rate.* Large initial learning rates will not result in missing out on the minimum during optimization, and can lead to quicker convergence.
- *Accelerate the learning rate decay.* As batch normalization makes a model more robust to hyperparameters, one can accelerate the rate at which the learning rate decays in addition to setting a higher initial learning rate for optimal training time.
- *Remove dropout.* One can get away with not using dropout layers when using batch normalization, since dropout can provide damage and/or slow down the training process. Batch normalization introduces an additional form of resistance to overfitting.
- *Reduce L2 weight regularization.* A huge part of training a neural network successfully is regularizing the weights. While using L2 regularization — penalizing objects at the square of their error to emphasize the punishment of larger objects — is effective and commonly used, it comes with its own set of problems, including an increase in weight complexity and tendency for overfitting. Batch normalization has regularizing properties, which may be a more ‘natural’ form of regularization.
- *Solving the vanishing gradient problem.* Although internal covariate shift may not improve accuracy, it is somewhat involved in the vanishing gradient problem. When the distribution of inputs shifts, it will fall prone to inherently smaller gradient updates due to the activation functions (for example, sigmoid, which puts miniscule values at anything ± 2.5 , or ReLU, which sets any $x < 0$ to 0). Batch normalization helps make sure that the signal is heard and not diminished by shifting distributions from the end to the beginning of the network propagation.

- *Solving the exploding gradient problem.* As batch normalization smooths the optimization landscape, it gets rid of the extreme gradients that accumulate, leading to the elimination of the major weight fluctuations that result from gradient build-up. This dramatically stabilizes learning.

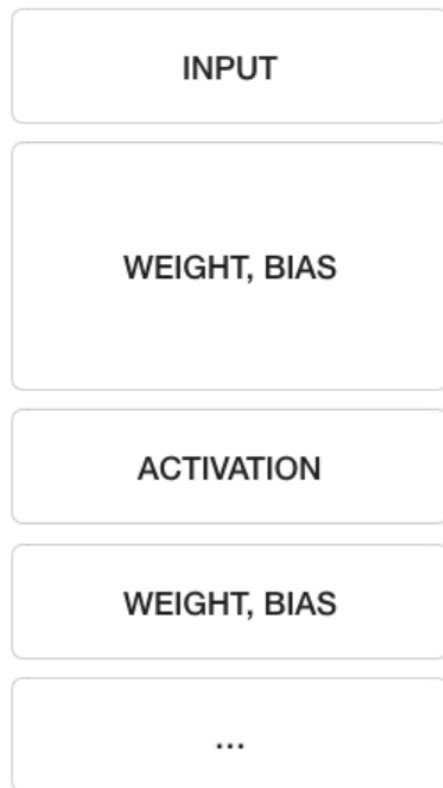
Deep Neural Network (DNN, 심층 신경망)

Vanishing Gradient vs Exploding Gradients

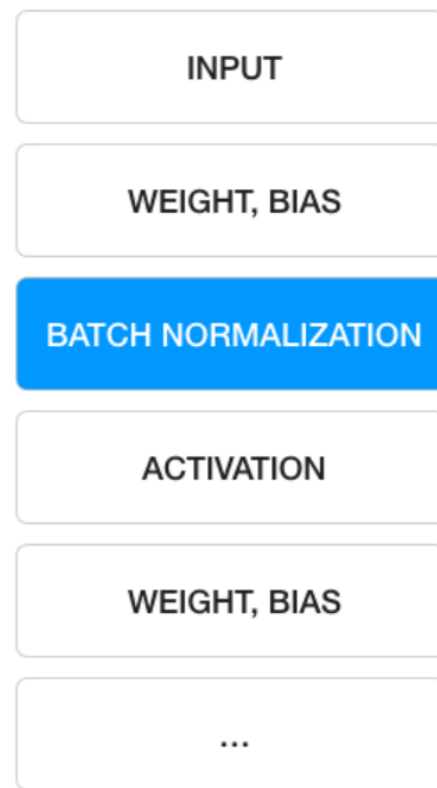
“Batch normalization

Activation 이전 ?? 이후?? 논쟁 중

DEFAULT ARCHITECTURE



BATCH NORM ARCHITECTURE



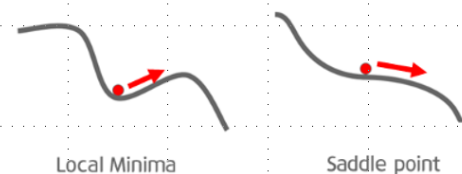
Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- ❖ Momentum
- ❖ L2 penalty
- ❖ Early stopping
- ❖ Dropout
- ❖ Data augmentation

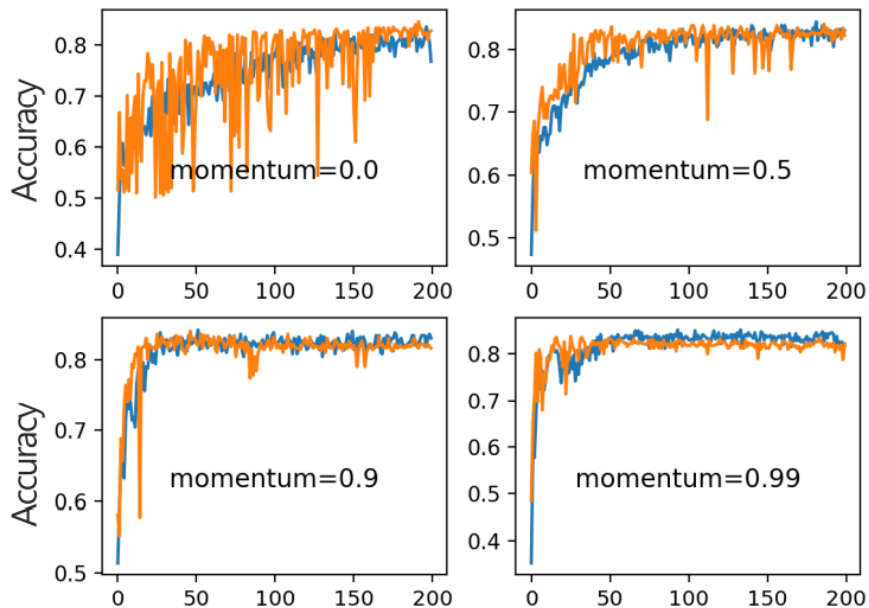
$$\hat{w}(t+1) := \hat{w}(t) - \alpha \cdot m_t \rightarrow \text{관성속도 (velocity)}$$
$$m_t = (\gamma \cdot m_{t-1} + \frac{\partial \text{Cost}(\hat{w}(t))}{\partial \hat{w}(t)})$$

Momentum



Gradient가 0이라 하더라도,
v값이 더해지면서 멈추지 않고 옆으로 이동할 수 있다!

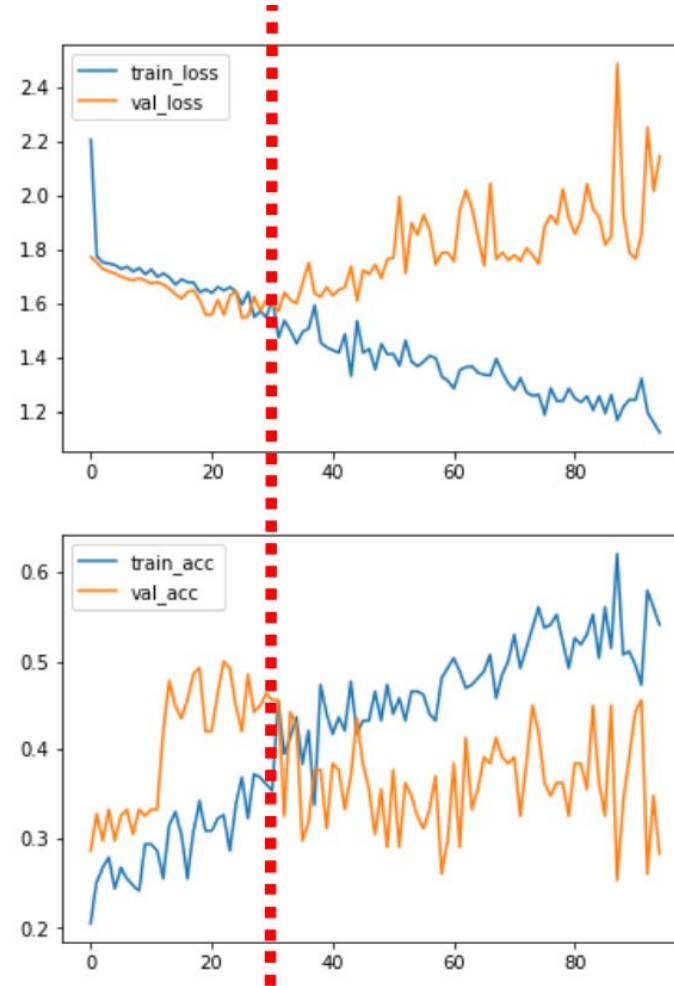
Local minimum에 빠지는 것 방지, 수렴 빠르게



Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- ❖ **Momentum**
- ❖ L2 penalty
- ❖ Early stopping
- ❖ Dropout
- ❖ Data augmentation



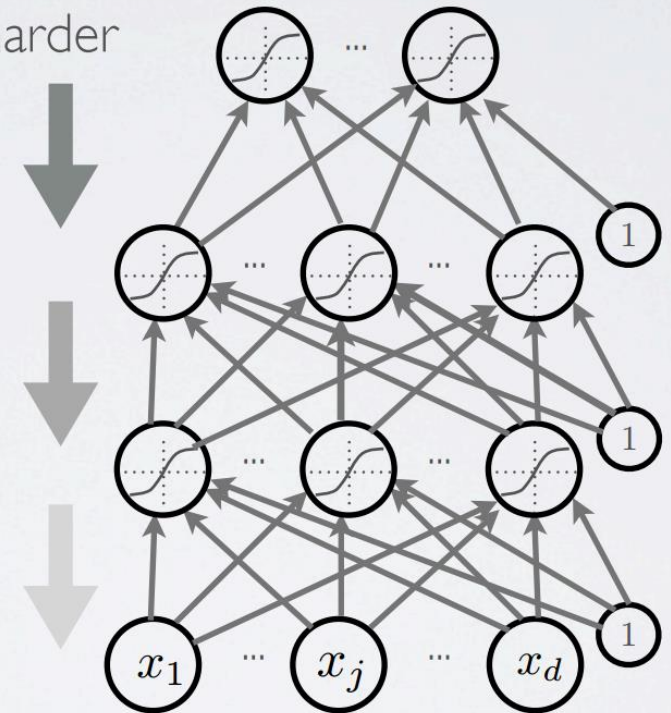
Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- ❖ **Momentum**
- ❖ L2 penalty
- ❖ Early stopping
- ❖ Dropout
- ❖ Data augmentation

Topics: why training is hard

- First hypothesis: optimization is harder (underfitting)
 - vanishing gradient problem
 - saturated units block gradient propagation
- This is a well known problem in recurrent neural networks



Convolution Neural Networks (CNN)

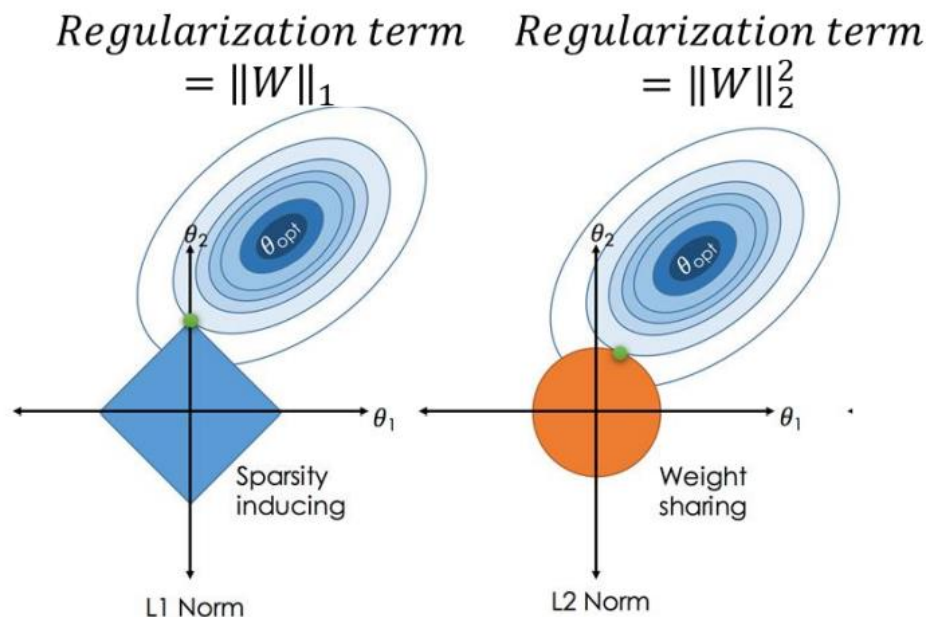
VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- ❖ **Momentum**
- ❖ L2 penalty
- ❖ Early stopping
- ❖ Dropout
- ❖ Data augmentation

모델의 파라미터 W 의 크기를 줄이는 제약을 추가

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \text{Loss}(y_i, f_{\theta}(x_i)) + \lambda \text{Regularization term}$$

Weight penalty
= weight decay
= regularization



Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

❖ **Momentum**

❖ L2 penalty

$$X_1 = 1, \quad X_2 = 1$$

❖ Early stopping

❖ Dropout

$$\text{Model 1} \quad +1 = 1000 \times X_1 - 999 \times X_2$$

❖ Data augmentation

$$\text{Model 2} \quad +1 = 2 \times X_1 - 1 \times X_2$$

$$X_1 = 1.01, \quad X_2 = 1.05$$

$$\text{Model 1} \quad -38.95 = 1000 \times X_1 - 999 \times X_2$$

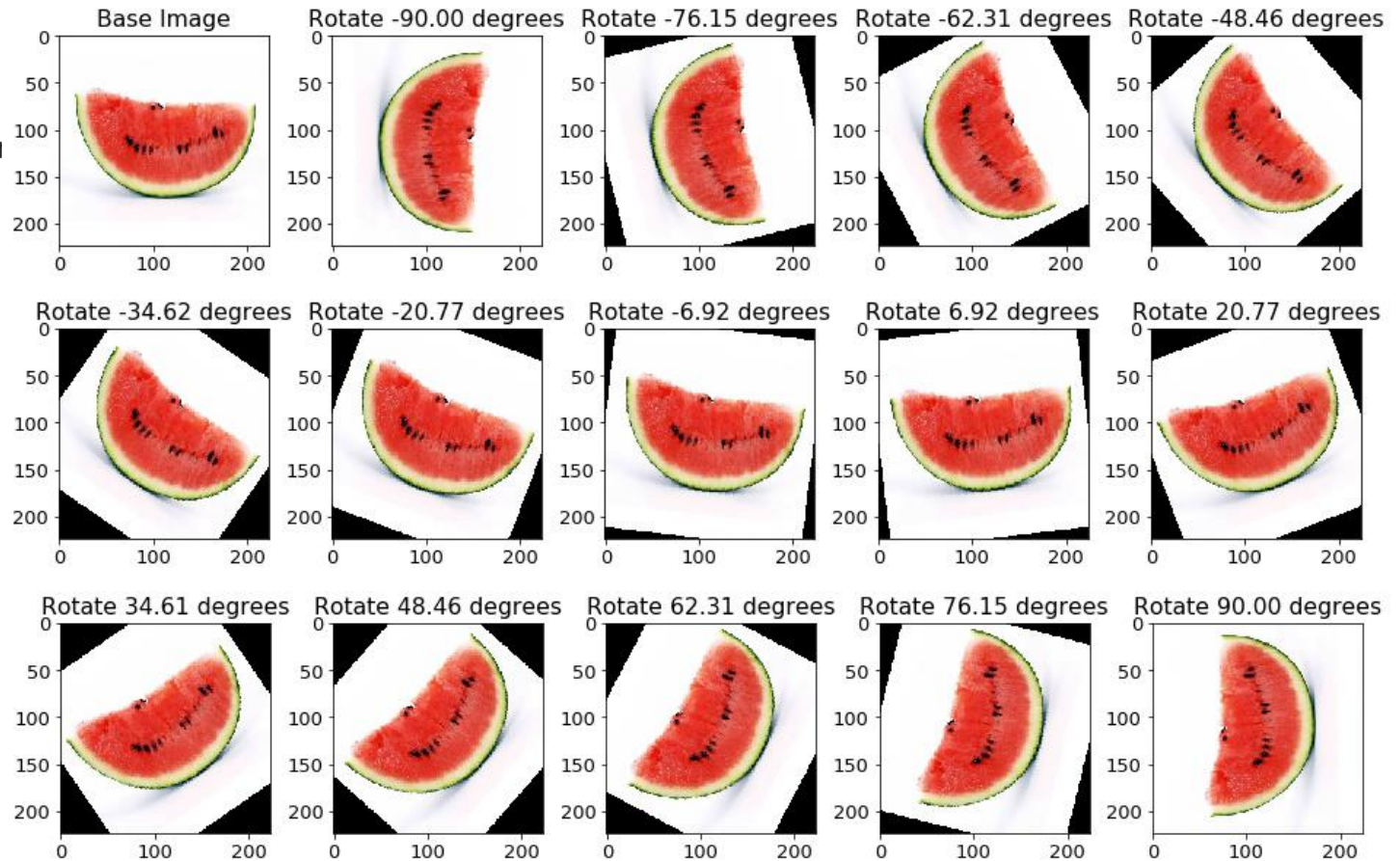
$$\text{Model 2} \quad +0.97 = 2 \times X_1 - 1 \times X_2$$

Convolution Neural Networks (CNN)

VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- ❖ Momentum
- ❖ L2 penalty
- ❖ Early stopping
- ❖ Dropout
- ❖ Data augmentation

Rotation



Convolution Neural Networks (CNN)

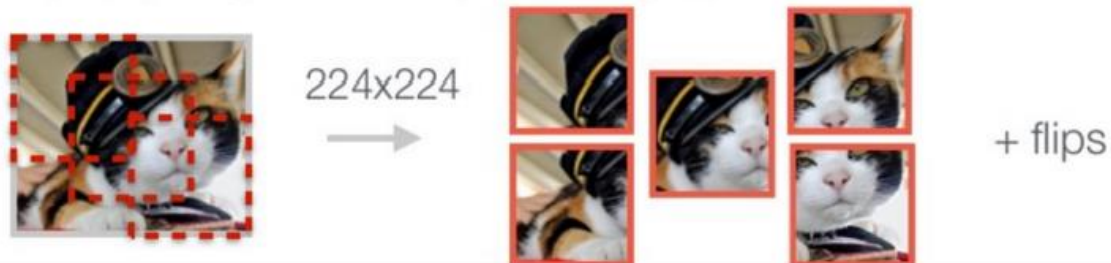
VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- ❖ Momentum
- ❖ L2 penalty
- ❖ Early stopping
- ❖ Dropout
- ❖ Data augmentation

• 좌우/상하 반전



c. Crop+Flip augmentation (= 10 images)

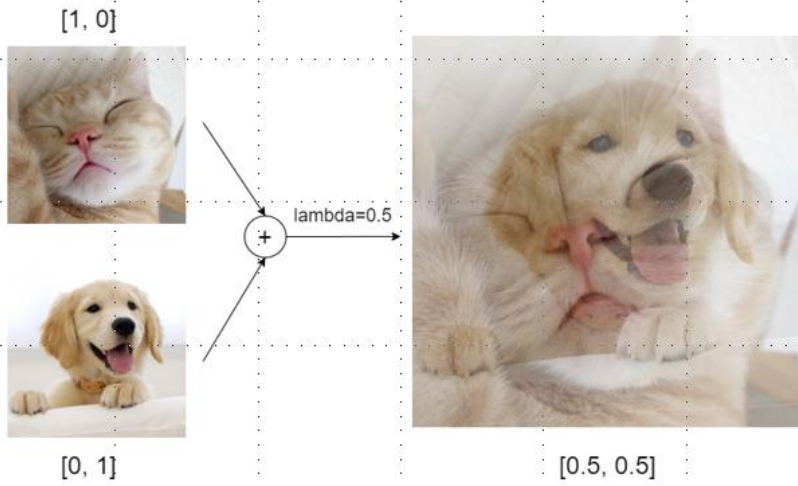


Convolution Neural Networks (CNN)

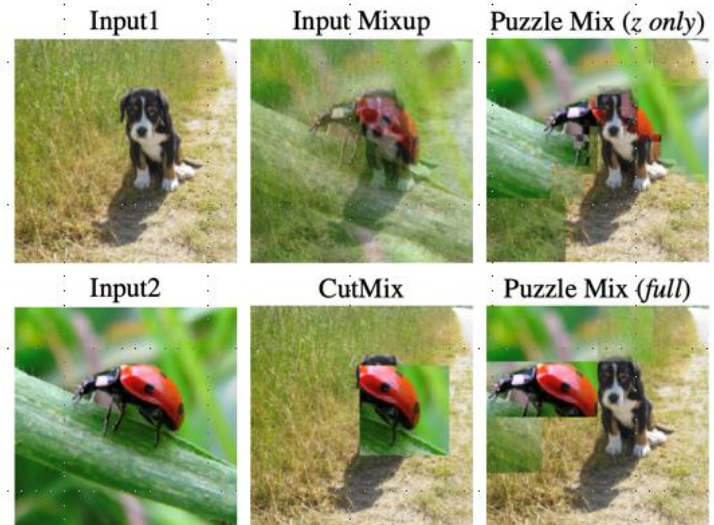
VGGNet Very deep convolutional networks
VGG stands for Visual Geometry Group, University of Oxford

- ❖ Momentum
- ❖ L2 penalty
- ❖ Early stopping
- ❖ Dropout
- ❖ Data augmentation

Mix up



Puzzle Mix



EOD